BIZM: Verification-Guided LLM Response Synthesis for Bounded East-West Threat Containment in Zero-Trust Microsegmentation

Hanh P. Du, Hung K. Luu, Minh T. Nguyen, Hoa N. Nguyen*

Department of Information Systems, VNU University of Engineering and Technology, Hanoi, 100000, Vietnam

Abstract

Zero-Trust microsegmentation limits lateral movement in data centers, yet its static policies cannot respond when a compromised workload abuses a permitted path. Fixed playbooks fail on unknown signatures, hand-written rules drift from the architecture they encode, and a large language model (LLM) on the enforcement path may hallucinate or follow injected instructions. This paper presents **BIZM** (**B**ounded-**I**solation **Z**ero-Trust **M**icrosegmentation), which treats east-west response as program synthesis under explicit safety invariants. Ahead of time, an LLM synthesizes a response program from the data-center knowledge base; an exhaustive verifier checks the bounded-isolation invariants on every point of the finite alert domain and returns counterexamples that guide repair; and a majority ensemble of verified programs is bound to the knowledge base it was synthesized from. The program answers covered alerts without a model call; an LLM agent reasons at runtime only over the rest. On both paths, deterministic validation and command-level Zero-Trust enforcement admit only source-faithful, time-bounded, DROP-only actions, so a wrong or manipulated model cannot widen its authority. On a SONiC spine-leaf testbed with Suricata monitoring, the synthesized program agrees with the knowledge base on all 14,970 graded points, against 96.1% for a hand-written contextual rule; the verifier rejected three of six initial programs, each repaired in one round; and resynthesis keeps 99.97% to 100% agreement after four knowledge-base changes. On live traffic, BIZM contains all threats without false positives, reduces median decision latency from 3.1 to 0.6 seconds, and produces no unsafe enforcement under the tested adversarial alerts.

Keywords: Zero-Trust, Microsegmentation, Lateral Movement, Large Language Models, Program Synthesis, Exhaustive Verification, Software-Defined Networking

1. Introduction

Modern data centers are dominated by east-west communication among internal services rather than by north-south traffic crossing the enterprise perimeter. Cloud computing, virtualization, microservices, and disaggregated service architectures mean that a single user-facing request may trigger many interactions across web, application, database, storage, and management services [1]. Once one workload is compromised, the internal service mesh becomes the primary surface for reconnaissance, privilege escalation, lateral movement, command-and-control communication, and data exfiltration. Perimeter-oriented defenses are therefore insufficient, because they provide limited visibility and control over service-to-service communication inside the data center.

Zero-Trust Architecture (ZTA) reduces this risk by removing implicit trust from the internal network and enforcing continuous verification, least-privilege access, and resource-centric policy control [2, 3]. Within ZTA, microsegmentation partitions workloads into fine-grained security zones and permits only explicitly authorized inter-zone communication, which demonstrably reduces attack-surface exposure [4, 5, 6, 7, 8]. Most deployed policies, however, are static: they are derived from baseline traffic or design-time dependency analysis and enforced until an operator revises them. A static policy blocks denied zone pairs, but it does not decide how to contain a compromised workload that abuses an otherwise permitted path.

East-west detection has advanced in parallel. Graph-based and temporal systems such as NetVigil and Euler detect anomalous internal communication and lateral movement at scale [9, 10]; signature-based engines such as Suricata provide explainable, low-latency alerts for known threats [11]; and AI-powered intrusion detection system (IDS) research has improved flow sensing, ensemble inference, and adversarial robustness [12, 13]. Detection alone, however, does not close the response loop. In operational settings, alerts are triaged by analysts or mapped to actions through security orchestration, automation, and response (SOAR) playbooks. Manual response is too slow for lateral-movement containment, whereas fixed playbooks generalize

*Corresponding author

Email addresses: hanhdp@vnu.edu.vn (Hanh P. Du), 22024546@vnu.edu.vn (Hung K. Luu), 23020628@vnu.edu.vn (Minh T. Nguyen), hoa.nguyen@vnu.edu.vn (Hoa N. Nguyen)

poorly to unknown signatures, adversarial alert content, and context-dependent permitted-path abuse.

This paper addresses the resulting *network-response problem*: given an east-west alert, decide whether temporary isolation is warranted, determine its scope, and enforce it without turning the decision component into an over-privileged network operator. A correct response depends on segmentation policy, topology, asset role, baseline communication, alert semantics, incident context, and the operational risk of blocking protected services. Deterministic rules can encode this context, but they are written for one version of the architecture and silently diverge from it as signatures, zones, and documented responses change. Large Language Models (LLMs) can combine heterogeneous context, follow structured schemas, invoke tools, and generate explanations [14, 15, 16, 17, 18, 19], but placing an LLM directly on the enforcement path is unsafe: it may hallucinate, misread adversarial alert text, emit malformed outputs, or target the wrong asset [20, 21, 22]. The challenge is therefore threefold: the response logic must handle permitted-path abuse, automated isolation must remain safe under uncertain or adversarial inputs, and the authority to change the fabric must remain deterministic, least-privileged, auditable, and fail-closed.

Our central observation is that most of the response logic is *determined by the architecture description* and does not change between alerts. For an alert that carries a cataloged signature on a known zone pair, the correct response follows from the policy matrix, the signature catalog, the protected-asset set, and the documented baselines. Reasoning about such an alert with an LLM at runtime is therefore both unnecessary and risky: it ties latency, cost, and availability to a model provider and exposes the enforcement path to alert text. We instead treat this logic as a *program to be synthesized*. The LLM reads the knowledge base once and writes a response program in a small, target-free language; because the program's input space is a finite domain of alert features, a verifier can check every bounded-isolation invariant on every input, return counterexamples to the model for repair, and accept only a program that is correct by construction on the whole domain. We call this *verification-guided response synthesis*. The verified program answers covered alerts in milliseconds without a model call and abstains where the knowledge base does not determine the answer; those residual alerts are reasoned about by an LLM agent at runtime, and both paths are bound by the same fail-closed validation and least-privilege enforcement contract.

This paper presents **BIZM** (Bounded-Isolation Zero-Trust Microsegmentation), a safety-constrained framework that implements this design on a SONiC-based spine-leaf fabric with Suricata monitoring. The core mechanism is *bounded isolation*: automated actions are restricted to source-faithful, provenance-marked, time-bounded DROP rules against the offending source, and every candidate decision is checked against fail-closed controls, including schema validation, prompt-injection sanitization, topology checking, protected-asset exclusion, off-target rejection, blast-radius limits, severity and confidence gates, and circuit breaking. The enforcement layer authenticates each request, binds it to a restricted agent identity, and authorizes the operation at command granularity. Safety is thus not delegated to the LLM; it is enforced by explicit invariants and least-privilege control, and on the compiled path it is established before any alert arrives.

The main contributions of this paper are as follows.

- **Problem formulation.** We formulate bounded isolation as a safety-constrained response-synthesis problem over a finite alert domain. We define a minimal action space and six admissibility invariants (action restriction, target fidelity, protected-asset exclusion, bounded scope, confidence gating, and provenance-bound authorization), prove that four of them hold architecturally regardless of model output, and show that all decision-level invariants of a synthesized program can be established exhaustively before deployment. We further prove that point-wise majority voting over verified programs preserves verification, which licenses ensembling without re-trusting the model.
- **Verification-guided response synthesis.** We introduce a synthesis loop in which an LLM translates the architecture knowledge base into a program of a target-free response language, an exhaustive verifier returns typed counterexamples that guide repair, a self-review step checks behavior against the knowledge base, and a strict-majority ensemble of verified programs is deployed and bound to the digest of the knowledge base it was synthesized from. Unlike LLM-with-verifier systems for configuration generation, the artifact verified here is the response decision itself, on every input it can receive.
- **Two-speed architecture.** We design BIZM, in which the synthesized program answers covered alerts and a graph-grounded LLM agent reasons over the residual alerts, with both paths bound by the same fail-closed validation and command-level least-privilege enforcement contract on a SONiC spine-leaf fabric.
- **Evaluation.** We implement BIZM on a software-defined networking (SDN) spine-leaf testbed with Suricata monitoring and evaluate exhaustive agreement with the knowledge base, the dynamics of the synthesis loop, recompilation under knowledge-base change, sensitivity to ensemble size and model choice, live closed-loop containment and latency, adversarial alert manipulation including a multi-source campaign, residual reasoning on unknown signatures, and verifier scalability, against signature-keyed, hand-written contextual, and guardrail-free LLM baselines.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formulates the bounded-isolation and response-synthesis problems and states the safety properties. Section 4 presents the BIZM architecture, the synthesis loop, and the runtime workflow. Section 5 describes the implementation and experimental design. Section 6 reports and discusses the results. Section 7 concludes the paper.

2. Related Work

We review prior work along five lines: Zero-Trust microsegmentation and east-west threat detection, LLM-assisted network operations, program synthesis with verifier feedback, automated threat response and programmable mitigation, and least-privilege enforcement for network automation. Table 1 positions BIZM against the closest systems.

2.1. Zero-Trust Microsegmentation and East-West Threat Detection

Microsegmentation is a core mechanism for realizing Zero-Trust in data centers. It partitions workloads into fine-grained security zones and restricts east-west communication to explicitly authorized service dependencies. Beyond NIST SP 800-207, recent surveys and guidance have refined Zero-Trust principles for cloud-native, distributed, and multi-location environments [3, 5, 6, 7], and NIST SP 800-204A provides microservices-oriented guidance for workload-level segmentation [8]. In most deployments, segmentation policies are derived from baseline traffic, role inference, or dependency analysis and enforced until an operator revises them. They reduce the attack surface but remain largely static: they can block denied zone pairs, yet they do not decide how to contain a compromised workload that abuses a permitted path.

Recent studies have automated segmentation-policy construction. Game of Zones derives security zones from high-level intents using Answer Set Programming [24]; ZT-SDN learns Zero-Trust access-control rules from graph-modeled network transactions and enforces them through an SDN controller [25]; Mani et al. infer endpoint roles from flow logs for public-cloud microsegmentation [23]; and Hari et al. adapt microsegmentation through a Bayesian trust score [32]. These works show that segmentation can be learned, synthesized, or adapted, but their object is the segmentation policy itself, not per-alert runtime containment under explicit safety invariants.

A parallel line detects east-west threats. NetVigil applies domain-specific graph neural networks to low-cost flow logs for data-center anomaly detection [9]; Euler models host activity as temporal graphs and detects lateral movement through anomalous link prediction [10]; self-supervised sequence models detect red-team lateral movement from authentication logs [33]; and AI-powered IDS research has improved traffic sensing, flow-level analysis, data augmentation, ensemble inference, and adversarial robustness [34, 35, 12]. These systems provide valuable signals, but they stop at detection or alert prioritization. They do not determine whether a specific source should be isolated, compute a bounded enforcement scope, or provide a least-privilege contract for applying a response to the fabric.

2.2. LLM-Assisted Network Operations

LLM-assisted network operations have attracted attention because language models can translate intent, retrieve technical context, generate structured outputs, and assist with configuration tasks. Dzeparoska et al. decompose application-management intents into policy-based execution steps [36]; NetConfEval shows both promise and fragility of LLMs for configuration and routing tasks [26]; LLNet translates intents into softwarized-device instructions with a small language model [27]; and Siino et al. integrate LLMs into network testbeds for automated experimentation [37]. General agent techniques, including tool use, self-consistency, and self-refinement, motivate structured reasoning pipelines [17, 18, 19, 20].

These studies bridge high-level intent and low-level network artifacts, but they address configuration generation, intent decomposition, device programming, or experiment automation, usually with an operator or a test workflow in the loop. BIZM places an LLM inside a security response loop, where an unsafe decision may disrupt production services. The model can therefore only propose a bounded isolation decision; deterministic validation and identity-aware command authorization decide whether the proposed action can reach the fabric. Correctness of generated text is not enough when the output may become a live network operation.

2.3. Program Synthesis with Verifier Feedback

Counterexample-guided inductive synthesis (CEGIS) alternates between a synthesizer that proposes a candidate program and a verifier that either accepts it or returns a counterexample that constrains the next proposal [38, 39]. Network verification has applied exhaustive and symbolic checking to forwarding behavior: Header Space Analysis checks reachability properties over the packet header space [40], and Batfish analyzes configurations against a model of the control plane [41]. More recently, Mondal et al. synthesize router configurations with an LLM, feed back the errors reported by syntax and semantic verifiers, and leave the remaining errors to a human [28].

BIZM is, in these terms, a CEGIS loop in which the LLM is the inductive synthesizer and exhaustive enumeration of a finite domain is the verifier. It differs from LLM-with-verifier systems for configuration generation in three respects. First, the artifact is the *response decision* rather than configuration text or a forwarding table, and the property verified is a set of safety invariants over the network effect of each decision. Second, because the

Table 1: Comparison of related work against BIZM by defensive-loop capability.

System	Core method	Response decision	Fabric enforcement	Bounded safety	Verified response logic	Closed loop
NetVigil [9]	Unsupervised GNN over east-west flow logs	No	No	No	No	No
Euler [10]	Temporal graph link prediction for lateral movement	No	No	No	No	No
Role-based microsegmentation [23]	Role inference for cloud microsegmentation policy	No	Design-time	No	No	No
Game of Zones [24]	Intent-based security-zone synthesis (ASP)	No	Design-time	No	Policy only	No
ZT-SDN [25]	Learned Zero-Trust rules in SDN	Policy learning	Yes	Partial	No	Partial
NetConfEval [26] / LLNet [27]	LLM-assisted network configuration	Config generation	No	No	No	No
Mondal et al. [28]	LLM router-configuration synthesis with verifier feedback	Config generation	No	Partial	Config only	No
DAERF [29] / P4-Secure [30]	Classifier-driven SDN/P4 mitigation	Coarse mitigation	Yes	No	No	Yes
Dynamic ZT zoning [31]	Stochastic-game policy adaptation	Policy adaptation	Yes	No	No	Yes
BIZM	Verification-guided LLM synthesis of a response program, bounded runtime reasoning on residual alerts	Yes	Yes	Yes	Yes (exhaustive)	Yes

Bounded safety is Partial when the system bounds the artifact it produces (syntactic and semantic errors of a configuration, or learned rules) but not the operational effect of a runtime action; *Verified response logic* is Yes only when the deployed decision logic is checked against explicit invariants before deployment.

program's input space is a finite domain of alert features, the verifier is exhaustive rather than symbolic or sampled: counterexamples are typed by the invariant they violate, and an accepted program carries a certificate on every input it can receive at runtime rather than lint-style feedback with residual human repair. Third, the loop closes without a human and ends in deploy-or-abstain. The specification is deliberately partial: the verifier establishes safety, whereas agreement with the documented response is measured against the knowledge base rather than verified, and a strict-majority ensemble of verified programs stands in for completeness of the synthesizer. The verifier also serves as a selective-prediction gate [42]: the program abstains where the knowledge base does not determine the response, and those alerts are deferred to the runtime agent rather than decided by a guess.

2.4. Automated Threat Response and Programmable Mitigation

Automated threat response follows three directions: rule-based playbooks, classifier-driven mitigation, and programmable data-plane defense. SOAR playbooks map alert types to predefined actions and are simple and auditable, but they generalize poorly to unknown signatures, ambiguous contexts, and permitted-path abuses that require topology and baseline reasoning. Incident-response guidance emphasizes that containment actions should be scoped, reversible, and auditable when they may affect business-critical services [43, 44, 45]. Machine-learning systems improve detection and prioritization, including under adversarial evasion and in malware settings [13, 46, 47, 48], but they output scores, classes, or coarse mitigation signals rather than bounded per-alert isolation decisions.

Several works couple detection and mitigation in SDN or programmable networks. DAERF integrates a hybrid deep autoencoder and random forest into an SDN mitigation framework [29]; P4-Secure performs in-band DDoS detection in the programmable data plane [30]; and P4DDLe extracts packet-level features in the data plane to improve intrusion detection while reducing control-channel load [49]. The programmable-networking literature provides mechanisms for installing and placing rules through OpenFlow, P4, ACL systems, and match-action pipelines [50, 51, 52, 53]. These mechanisms improve how and where enforcement can be realized, but they do not decide whether an automated isolation action is admissible under Zero-Trust safety constraints.

Other systems adapt detector or policy logic at runtime.

Auto-IDaS uses reinforcement learning to configure IDS models, tasks, and capacities [54]; Bello and Refaey Hussein adapt Zero-Trust security zoning through stochastic games and meta-learning [31]. These approaches operate at a different abstraction level: Auto-IDaS optimizes the detection service, and dynamic zoning adapts policy in mobile-edge settings. BIZM evaluates an individual east-west alert against a data-center segmentation baseline and applies only a constrained, auditable, temporary source-isolation rule.

2.5. Least-Privilege Enforcement for Network Automation

Autonomous network response is unsafe unless the response component is itself constrained. This issue is acute when LLMs are used in security automation, because the model may hallucinate, be manipulated by adversarial alert text, or generate syntactically valid but operationally unsafe actions. Prior work on Zero-Trust, SDN security, and intent-based management highlights identity verification, policy enforcement, conflict detection, and protocol-aware control [3, 55], but these mechanisms are designed for operator commands, service intents, or static policy deployment, not for bounding the authority of an autonomous response component.

Policy-enforcement and configuration-synthesis systems address related but different problems. I2NSF policy-conflict detection prevents inconsistent security policies before deployment [56]; automated firewall synthesis generates conflict-free configurations for virtual networks [57]; and ACL placement methods optimize where rules should be installed under hardware constraints [58]. These works assume that a policy or intent has already been accepted as legitimate. They do not ensure that an automatically generated isolation command is within the agent's authority, targets only the alert source, avoids protected assets, and carries a bounded lifetime and provenance mark.

Prompt injection is a systematic threat to LLM-integrated applications [22], and guardrail frameworks constrain agents at the model and tool boundary: NeMo Guardrails adds programmable rails that filter model inputs and outputs [59], StruQ separates trusted instructions from untrusted data [60], and IsolateGPT isolates the execution of third-party LLM applications [61]. These controls run in the agent host and decide what the model reads or whether a tool call may proceed. The invariants of Section 3.4 differ in where and on what they are enforced: they constrain the network effect of an action, namely its type, target, lifetime, and protected-asset reach, and they are checked again by the Secure Framework and the switch management channel, outside the agent process. A compromised or misled agent can therefore at most fail to contain, never widen its authority over the fabric.

2.6. Research Gap

The reviewed literature leaves a common gap. Existing systems improve segmentation design, east-west detection, LLM-assisted configuration, programmable mitigation, or policy enforcement, but none jointly provides contextual per-alert reasoning, explicit isolation invariants, protected-asset safety, provenance, and command-level least privilege in a single closed response loop, and none derives a *verified* deterministic response tier from the architecture description. BIZM addresses this gap by framing adaptive microsegmentation response as a bounded-isolation problem in which the LLM synthesizes verified response logic ahead of time and proposes candidate actions at runtime, while the surrounding architecture controls admissibility, target, scope, and enforcement authority. Because the closest systems operate on different infrastructures and occupy different stages of the defensive loop, a direct numerical comparison would be misleading; Table 1 therefore positions BIZM by capability rather than by absolute performance.

Table 2: Notation.

Symbol	Meaning
Z, P	Security zones; static policy $P : Z \times Z \to \{\text{ALLOW}, \text{DENY}\}$
B	Legitimate communication model on permitted pairs
$\mathcal{N}$	Immutable set of protected assets
$a = (s, d, \rho, \pi, \sigma, \ell, \kappa, \tau, t)$	Alert: source, destination, protocol, port, signature, severity, category, text, time
$H(s)$	Recent alert history of source s
$u = \langle \alpha, \theta, \rho, \pi, \delta, c, m \rangle$	Decision: action, target, protocol, port, TTL, confidence, ATT&CK class
$\mathcal{A}(a, u)$	Admissibility predicate (invariants I1 to I6)
K, $h(K)$	Knowledge-base snapshot and its digest
$D(K)$, $R(K)$	Finite alert-feature domain; realizable sub-domain
ϕ_K	Total feature map from alerts and history to $D(K)$
$\mathcal{G}$	Target-free response language
Π, $\Pi^{\oplus}$	Response program; strict-majority ensemble
$\mathcal{V}_K(\Pi)$	Counterexample set returned by the verifier
τ_c, τ_Π	Confidence thresholds (runtime path; compiled path)
$[\delta_{\min}, \delta_{\max}]$, $\mathcal{L}$	TTL bounds; blast-radius limits
k, $\eta_{\max}$	Ensemble size; maximum repair rounds

3. Problem Formulation

Table 2 summarizes the notation used throughout the paper.

3.1. System and Network Model

We consider a data-center fabric partitioned into a set of security zones $Z = \{z_1, \ldots, z_n\}$. Each zone groups workloads with a common operational role, such as web, application, database, monitoring, or management. The zones are interconnected through a spine-leaf fabric, where leaf switches serve as the primary enforcement points because they are closest to the workloads. A static microsegmentation policy is represented as

$$P : Z \times Z \to \{\text{ALLOW}, \text{DENY}\},$$

with default decision DENY. A zone pair is permitted only if it is explicitly listed in P. The policy is compiled into data-plane rules on the leaf switches, so traffic on denied zone pairs is blocked independently of the response system.

A network flow is modeled as $f = (s, d, \rho, \pi, \phi)$, where s and d are the source and destination addresses, ρ is the transport protocol, π is the destination service port, and ϕ denotes behavioral features such as connection rate, byte volume, session duration, periodicity, and request pattern. Let $z(s)$ and $z(d)$ denote the zones of the endpoints. A flow is statically permitted if $P(z(s), z(d)) = \text{ALLOW}$. Static permission does not imply benign behavior, so we define a legitimate communication model

$$B \subseteq \{f \mid P(z(s), z(d)) = \text{ALLOW}\},$$

which captures expected communication on permitted zone pairs, including service ports, directionality, traffic volume, and temporal behavior. In BIZM, B is part of the architecture knowledge base and is maintained by the operator rather than learned online. Each baseline flow records its zones, hosts, port, protocol, cadence, and expected volume; each flow type carries a mean rate, a 95th-percentile rate, and anomaly thresholds at three and ten times the mean; and a list of permitted-path abuse patterns names the signatures that detect them. A change to B changes the knowledge-base digest and therefore triggers recompilation (Section 4.3).

An east-west IDS observes mirrored inter-zone traffic and emits alerts for policy violations, signature matches, or suspicious behavioral patterns. An alert is represented as

$$a = (s, d, \rho, \pi, \sigma, \ell, \kappa, \tau, t),$$

where σ is the signature identifier, ℓ is the severity level, κ is the alert category, τ is the textual signature or alert description, and t is the timestamp. The IDS is the source of suspicious evidence. The problem addressed here is not raw traffic detection but response-level decision making: given an alert, decide whether bounded source isolation is warranted and whether the resulting action can be safely enforced.

3.2. Threat Model

We assume an adversary that has compromised one or more internal workloads and operates over east-west traffic. The adversary may perform reconnaissance, lateral movement, privilege escalation, command-and-control communication, destructive queries, or data exfiltration. We consider two primary classes of malicious behavior.

The first class is a *policy violation*: a compromised workload initiates traffic on a denied zone pair, such as a web host connecting directly to a database. The static policy may already block the violating packets, but the attempt indicates a potentially compromised source, and the response system must decide whether that source should be temporarily isolated.

The second class is *permitted-path abuse*: a compromised workload uses an allowed zone pair but deviates from the legitimate communication model B, for example by high-rate extraction, destructive commands, or abnormal request sequences over a permitted database connection. Static microsegmentation cannot block such behavior without disrupting legitimate service traffic, because the zone pair and service port remain valid. The response decision must account for baseline behavior, asset role, zone relationship, and the operational impact of isolation.

The adversary may also attack the response layer. Alert fields τ and κ may contain attacker-influenced text, including prompt-injection instructions that attempt to redirect isolation to a protected management host, an IDS component, or an unrelated workload; the source field may reflect spoofing, detector error, or incomplete telemetry; and many compromised sources may act in concert to exhaust response capacity. We therefore distinguish transport validity from semantic trust: the alert transport is assumed to be syntactically available, but alert content is untrusted input to the decision component. The decision component is also untrusted for enforcement: it may hallucinate, misinterpret context, produce malformed output, or select an unsafe target. No model output reaches the fabric unless it satisfies deterministic safety checks and least-privilege authorization. Target fidelity (I2 below) binds the enforcement target to the header source, not to the true origin of the traffic: an attacker who can spoof a non-protected address can at most cause a bounded, time-limited drop of that address's connections. This availability vector is bounded by the blast-radius limits rather than excluded by the invariants, and we return to it in Sections 6.5 and 6.9.

We trust the enforcement layer to implement authentication, authorization, provenance stamping, and command-level rejection correctly. We do not consider compromise of the switch operating system, of the enforcement implementation, or of the IDS engine, inspection of encrypted payloads beyond what the IDS observes, or north-south traffic. These are important deployment concerns but lie outside the bounded east-west response problem studied here.

3.3. Bounded-Isolation Decision Problem

Given an alert a, the system produces a response decision u and, when warranted, enforces it as a temporary

data-plane rule. A decision is

$$u = \langle \alpha, \theta, \rho, \pi, \delta, c, m \rangle,$$

where α is the action, θ the enforcement target, ρ and π the protocol and destination port, δ the rule time-to-live, c the decision confidence, and m the MITRE ATT&CK classification [62, 63]. The action space is intentionally minimal, $\alpha \in \{\text{DROP}, \text{NONE}\}$: DROP creates a temporary isolation rule against the offending source, whereas NONE denotes logging, audit recording, or human referral without automated enforcement. The synthesized program of Section 3.5 refines NONE into LOG_ONLY, a decided non-action, and ABSTAIN, a deferral to the runtime path; both map to $\alpha = \text{NONE}$ at the enforcement boundary.

The objective is to maximize correct bounded containment while limiting closed-loop response latency. Let $y(a) \in \{\text{THREAT}, \text{BENIGN}\}$ denote the ground-truth status of the alert context, let $\text{Containment}(a, u) = 1$ if u is an admissible DROP and $y(a) = \text{THREAT}$ or u is NONE and $y(a) = \text{BENIGN}$, and 0 otherwise, let $\text{Latency}(a, u)$ be the time from alert ingestion to enforcement, and let $\mathcal{A}(a, u)$ be an admissibility predicate that holds only when u satisfies the invariants of Section 3.4. The response problem is

$$\max_{u} \ \text{Containment}(a, u) - \lambda \, \text{Latency}(a, u), \quad \text{s.t. } \mathcal{A}(a, u) = \text{TRUE}, \tag{1}$$

where $\lambda \geq 0$ weights latency against correctness. The optimization form captures the design goal; the implementation enforces the constraint through explicit invariants rather than through unconstrained model behavior, and optimizes the objective by answering determined alerts from a synthesized program and the rest from a reasoning engine.

3.4. Bounded-Isolation Invariants

A decision u is admissible only if it satisfies the following invariants.

- **(I1) Action restriction.** $\alpha \in \{\text{DROP}, \text{NONE}\}$. Automated response may create a temporary DROP rule or take no enforcement action; it must not create an ACCEPT rule, delete another entity's rule, modify unrelated configuration, or perform general network-management operations.
- **(I2) Target fidelity.** $\theta = s$. The blocked address must be the alert source as a single host address; a prefix that merely contains the source is rejected. This prevents hallucinated, injected, or off-target isolation.
- **(I3) Protected-asset exclusion.** $\theta \notin \mathcal{N}$, where $\mathcal{N}$ is an immutable protected set that includes control-plane components, IDS components, zone gateways, management services, and monitoring services. These assets cannot be isolated automatically even if an alert names them as the source. A rule scoped to a non-protected source may still restrict that source's access to a protected destination, because it drops only connections the source initiates.
- **(I4) Bounded scope.** Every enforced DROP rule carries a time-to-live $\delta \in [\delta_{\min}, \delta_{\max}]$ and satisfies blast-radius limits $\mathcal{L}$ on the number of rules installed per minute, the number of active automated rules, and the number of rules per source within a time window.
- **(I5) Severity and confidence gating.** Automatic enforcement is allowed only when $\ell \in \{\text{P1}, \text{P2}\}$ and $c \geq \tau_c$. Lower-severity, low-confidence, ambiguous, or incomplete cases are logged, audited, or referred for human review.
- **(I6) Provenance and least-privilege authorization.** Every enforced rule carries a provenance mark that binds it to the automated response path and is authorized under a least-privilege role at command granularity. The response component is authorized only to create bounded DROP rules; unauthenticated, over-privileged, or unsupported operations are rejected before reaching the fabric.

Invariants I1, I2, I3, and I6 are architectural: they must hold regardless of the model output. Invariants I4 and I5 are deployment policies that bound operational risk and can be tuned by administrators. Together they define the bounded action contract exposed to the decision component.

3.5. Response Synthesis Problem

The runtime decision problem above is answered per alert. The compiled path of BIZM instead answers it for a whole class of alerts at once, which requires a finite abstraction of the alert and a program over that abstraction.

Definition 1 (Feature abstraction). Let K be a knowledge-base snapshot with signature catalog Σ_K, zone set Z, and protected set $\mathcal{N}$. The alert-feature domain is

$$D(K) = (\Sigma_K \cup \{\bot\}) \times L \times (Z \cup \{\text{ext}\})^2 \times \{0, 1\} \times \mathcal{H},$$

where $L = \{\text{P1}, \ldots, \text{P4}\}$ is the severity scale, the two zone coordinates are the source and destination zones, the bit marks a protected source, and $\mathcal{H}$ is the set of consistent history tuples (prior alerts of the source $\in \{0, 1+\}$, other targets of the source $\in \{0, 1, 2+\}$, other ports on the target $\in \{0, 1+\}$); a source with no prior alert has no other targets or ports, so $|\mathcal{H}| = 1 + 2 \cdot 3 = 7$. The policy verdict (ALLOW, DENY, intra-zone, external) is a derived feature of the zone pair under P. The feature map ϕ_K sends an alert a and the history $H(s)$, taken over a fixed window, to a point of $D(K)$ and is *total*: a signature outside the catalog maps to $\bot$, an address outside the fabric to ext, and a severity outside P1 to P4 to P4.

The realizable sub-domain $R(K) \subseteq D(K)$ contains the points the deployed IDS can emit, that is, a cataloged signature with its cataloged severity on a zone pair its detection logic covers. Verification is performed on all of $D(K)$ so that spoofed or internally inconsistent alerts are covered; agreement with the knowledge base is graded on $R(K)$.

Definition 2 (Response language and program). A program Π of the response language $\mathcal{G}$ is an ordered list of rules $\langle r_1, \ldots, r_n \rangle$. Each rule $r_i = (\varphi_i, o_i)$ has a guard φ_i, a conjunction of membership literals over the coordinates of $D(K)$, and an outcome $o_i \in O = \{\text{DROP}(scope, \delta, c, cite), \text{LOG_ONLY}, \text{ABSTAIN}\}$, where $scope \in \{\text{flow}, \text{pair}, \text{host}\}$ selects the destination and port matched by the rule and *cite* is the set of knowledge-base entries the rule relies on. The semantics is first-match: $\Pi(x) = o_i$ for the least i with $x \models \varphi_i$, and $\Pi(x) = \text{ABSTAIN}$ if no guard holds. The language has no target field: an enforced DROP always blocks the alert source as a single host.

Because every guard is evaluated on a point of $D(K)$ and the domain is finite, Π is a total function $D(K) \to O$ and can be tabulated.

Definition 3 (Verifier). The verifier $\mathcal{V}_K$ evaluates Π on every $x \in D(K)$ and returns the set of counterexamples $\mathcal{V}_K(\Pi) = \{(x, r_i, I)\}$ such that $\Pi(x)$ is decided by rule r_i and violates check I, where I ranges over: a DROP on a protected or external source (I3); a TTL outside $[\delta_{\min}, \delta_{\max}]$, host scope outside P1 or with $\delta > 1800$ s, or a DROP that overrides a permitted path with anything but flow scope, $\delta \leq 1800$ s, and $c \geq 0.90$ (the TTL and scope component of I4, written I4^{s}); a DROP outside P1 and P2 or with $c < \tau_\Pi$ (I5); and a DROP on an unknown signature or without a knowledge-base citation (check C, the compile-time counterpart of the provenance requirement in I6). A program is *verified* iff $\mathcal{V}_K(\Pi) = \emptyset$. The rate components of I4 (rules per minute, active rules, rules per source) and the authorization component of I6 depend on runtime state and are enforced by the shared validation and enforcement stage, not by the verifier.

The verifier is sound and complete for the checks it performs: every check is a function of the point x and the decision parameters alone, and every point of $D(K)$ is examined, so $\mathcal{V}_K(\Pi) = \emptyset$ iff no alert can make Π violate I3, I4^{s}, I5, or C. It is deliberately *not* a specification of correctness: agreement with the documented response is not a safety property and is measured, not verified. The *response synthesis problem* is then: given K, produce a verified program Π that agrees with the response documented in K on every graded point of $R(K)$ and abstains wherever K does not determine the response. Agreement is measured against ground truth derived from K by a fixed grading procedure that is independent of any program (Section 5.4); the grader answers only whether a graded point should be contained, whereas the program must also choose scope, TTL, confidence, and citations and must satisfy the invariants on all of $D(K)$, which is why the grader cannot itself serve as the response program. The trade-off between coverage and abstention is a property of the synthesized program rather than of the verifier.

3.6. Safety Properties

The invariants provide a safety-by-construction guarantee for the most critical unsafe response classes.

Proposition 1 (Architectural safety). *If the validation and enforcement layers are correctly implemented, no rule installed by BIZM can violate I1, I2, I3, or I6, regardless of the output produced by the LLM decision component.*

PROOF. I1 holds because the enforcement layer accepts only rule-creation requests whose action is DROP; ACCEPT, deletion, and unrelated configuration operations are outside the authorized action set and are rejected. I2 holds because the validator checks that the requested target equals the alert source before provenance stamping or enforcement. I3 holds because $\mathcal{N}$ is maintained outside the decision component and checked after model output but before rule installation; any target in $\mathcal{N}$ is rejected regardless of the model rationale. I6 holds because every accepted request is authenticated, bound to a restricted response role, authorized at command granularity, and stamped with provenance metadata before reaching the fabric. Malformed, injected, hallucinated, or over-privileged outputs can therefore only result in rejection or non-enforcement.

Proposition 1 does not claim that the model selects the optimal response. It establishes the narrower property that even when the model is wrong, its authority remains bounded: it may fail to recommend isolation or produce a low-confidence decision, but it cannot install an arbitrary or protected-asset-disrupting rule.

The compiled path admits a stronger property, because its decisions can be checked before any alert arrives.

Proposition 2 (Exhaustive decision safety). *Let K be a knowledge-base snapshot and Π a program with $\mathcal{V}_K(\Pi) = \emptyset$. While the deployed knowledge base equals K, every decision issued by Π satisfies I1, I2, I3, I4^{s}, and I5 for any alert, including alerts with spoofed or internally inconsistent fields.*

PROOF. I1 and I2 hold by construction of $\mathcal{G}$: outcomes are limited to DROP, LOG_ONLY, and ABSTAIN, and there is no target field, so an enforced rule always names the alert source. Because ϕ_K is total, every alert and history maps to a point of $D(K)$. The verifier has evaluated Π on all of $D(K)$ and found no decision violating I3, I4^{s}, or I5, so no alert can produce one. The compiled path is enabled only when the digest of the deployed knowledge base equals $h(K)$ and Π passes verification again at load time.

The rate components of I4 and the authorization of I6 are enforced for program decisions by the same runtime stage as for agent decisions, so Proposition 1 continues to cover them. Together, the two propositions mean that a program decision is checked twice, once exhaustively ahead of time and once at the enforcement boundary.

Sampling the synthesizer produces programs that differ in coverage and in individual errors. BIZM therefore deploys an ensemble, and the following lemma shows that doing so does not require re-trusting the model.

Lemma 1 (Closure under majority voting). *Let $\Pi_1, \ldots, \Pi_k$ be programs with $\mathcal{V}_K(\Pi_j) = \emptyset$ for all j. Define $\Pi^{\oplus}$ point-wise on $D(K)$: if a strict majority of the Π_j agree on the outcome type at x, $\Pi^{\oplus}(x)$ takes that type; otherwise $\Pi^{\oplus}(x) =$ ABSTAIN. For a majority DROP, the scope is the narrowest among the agreeing members, the TTL and confidence are the minima among them, and the citation set is their union. Then $\mathcal{V}_K(\Pi^{\oplus}) = \emptyset$.*

PROOF. Only DROP outcomes can violate an invariant. Let $\Pi^{\oplus}(x) =$ DROP and let J be the agreeing members. Every check performed by $\mathcal{V}_K$ on a DROP at x is a conjunction of (i) a predicate on x alone (not protected, not external, cataloged signature, severity in P1 or P2, and, for the permitted-path and host-scope checks, the policy verdict and severity of x), and (ii) bounds that are monotone in the decision parameters: an upper bound on scope under a scope ordering flow $\prec$ pair $\prec$ host, an upper bound on δ together with the lower bound $\delta_{\min}$, a lower bound on c, and non-emptiness of the citation set. Part (i) holds because it holds for every $j \in J$, each of which is verified and issues a DROP at x. Part (ii) holds because each $j \in J$ satisfies the same bounds at x, and the narrowest scope, the minimum of values that each satisfy an upper bound (respectively, exceed $\delta_{\min}$), the minimum of values that each satisfy a lower bound, and the union of non-empty sets satisfy them as well. Hence no counterexample exists at x, and $\mathcal{V}_K(\Pi^{\oplus}) = \emptyset$.

Lemma 1 is a soundness result for the ensemble construction; BIZM nevertheless re-verifies $\Pi^{\oplus}$ as a whole, so that the deployed artifact never depends on the correctness of the ensemble code.

Proposition 3 (Cost and termination). *Let*

$$|D(K)| = (|\Sigma_K| + 1) \cdot |L| \cdot (|Z| + 1)^2 \cdot 2 \cdot |\mathcal{H}|.$$

One verification evaluates at most $|\Pi|$ guards on each point and costs $O(|D(K)| \cdot |\Pi|)$ guard evaluations; the check is embarrassingly parallel across signatures. A synthesis run with ensemble size k and at most $\eta_{\max}$ repair rounds per member issues at most $k(\eta_{\max}+2)$ model calls and $k(\eta_{\max}+2)+1$ verifications, and therefore terminates.

The totality of ϕ_K buys exhaustive coverage of every runtime input, but it also fixes what the program can see. Payload evidence, the behavioral features ϕ of a flow, the numeric thresholds of B, multi-hop pivot structure, and alert text are outside $D(K)$: the compiled path recognizes permitted-path abuse only through the content signatures that B names for it, and the guarantee is relative to the fixed history window, destination-port and protocol binding, and thresholds under which $D(K)$ was defined. A program that needs evidence outside $D(K)$ must abstain, and those alerts are deferred to the runtime path. This is the intended division of labor, and Section 6.6 measures its cost.

3.7. Design Requirements

The system and threat models lead to eight requirements, which Sections 4 and 6 trace back to.

- **(R1) Closed-loop bounded containment:** a severe east-west alert is converted into an enforced temporary isolation rule without human intervention when the decision is confident and all invariants hold.
- **(R2) Contextual decision-making:** the system reasons beyond a fixed signature-to-action map, handles permitted-path anomalies, unknown signatures, and benign high-volume communication, and follows the architecture knowledge base as it changes without hand-maintained decision tables.
- **(R3) Fail-closed safety:** malformed output, prompt injection, off-target action, protected-asset targets, low-confidence decisions, policy conflicts, and validation errors result in non-enforcement.
- **(R4) Least-privilege enforcement:** the response component is an untrusted subject bound to a role whose only automated authority is to create temporary DROP rules within the bounded action contract.
- **(R5) Auditability and provenance:** every decision records the alert, retrieved context, candidate decision, validation result, reasoning trace or program rationale, MITRE mapping, TTL, provenance mark, and enforcement outcome.
- **(R6) Substitutability:** the decision model, IDS, and network devices are replaceable behind stable interfaces.
- **(R7) Bounded operational impact:** time-bounded rules, protected-asset exclusions, per-source and global rate limits, and circuit breaking limit the blast radius of automation.
- **(R8) Predictable decision path:** alerts whose response is determined by the knowledge base are decided without a model call, so that latency, cost, and availability for known threats do not depend on the model provider.

4. Proposed Method: BIZM Architecture and Verification-Guided Response Synthesis

BIZM is a bounded-response architecture for Zero-Trust microsegmentation in a SONiC-based data-center fabric. Its goal is to close the loop from east-west alerting to temporary source-side containment without giving the LLM direct authority over the network. The LLM is used at two speeds: ahead of time, as the synthesizer of a verified response program, and at runtime, as a reasoning engine over the alerts the program does not cover. Neither the program nor the runtime agent issues network commands; each produces a candidate bounded-isolation decision, and deterministic validation and identity-aware control-plane mediation decide whether it may become a device-level DROP rule.

BIZM is organized into four planes. The *data plane* forwards workload traffic, enforces static segmentation policy, mirrors selected east-west traffic to the IDS, and installs temporary DROP rules on the SONiC fabric. The *decision plane*, implemented as the LLM Security Agent, answers covered alerts from the synthesized program and, for the remaining alerts, classifies them, retrieves graph-grounded context, and performs LLM-based reasoning; it then validates candidate decisions and generates policy/YANG-oriented bounded-response requests. The *control plane* mediates enforcement through the IDS Agent, the Secure Framework, and ONAP SDN-C. The *operator layer* provides live visibility into alerts, flows, candidate decisions, validation outcomes, and enforcement status. This separation keeps contextual reasoning, safety validation, and command-level enforcement under distinct control responsibilities.

4.1. Architecture Overview

Figure 1 shows the overall architecture and its six-step message flow. First, the SONiC spine-leaf fabric mirrors selected east-west traffic to Suricata through SPAN or ERSPAN. Second, Suricata emits EVE JSON alerts, which reach the IDS Agent and the LLM Security Agent over Server-Sent Events (SSE). Third, the LLM Security Agent answers the alert from the synthesized program or, when it abstains, classifies it, retrieves graph-grounded context, and reasons over it, and then validates the candidate decision. Fourth, an admissible DROP request is pushed to the IDS Agent. Fifth, the IDS Agent forwards it over REST to the Secure Framework, which authorizes it under mutual TLS (mTLS) and role-based access control (RBAC). Sixth, the accepted rule is enforced through ONAP SDN-C and the southbound gRPC Network Management Interface (gNMI) path. Telemetry flows upward from the data plane to the decision plane, whereas policy commands flow downward only after validation and identity-aware mediation. Alerts and flow updates are also streamed to the operator dashboard.

The architecture provides two safety barriers. The first is the fail-closed validation path inside the LLM Security Agent, which rejects malformed decisions, low-confidence responses, off-target rules, protected-asset violations, TTL violations, and blast-radius violations before any control-plane request is generated. The second is the identity-aware enforcement path in the control plane, which restricts accepted requests to authorized DROP-only operations and installs them through a mutually authenticated management channel. The LLM may contribute contextual reasoning, but it cannot acquire arbitrary network-management authority.

4.2. Data Plane and Monitoring

The data plane is a SONiC-based spine-leaf fabric. Leaf switches are the closest enforcement points to workloads and host the temporary source DROP rules. BIZM assumes a deny-by-default segmentation baseline: inter-zone communication is allowed only when explicitly permitted by the policy matrix, and all other east-west paths are blocked. Runtime containment is applied at the source-side leaf switch, which limits the scope of the response and avoids fabric-wide disruption.

Figure 2 shows the prototype topology. Workloads are grouped into eight zones: WEB, APP-GW, APP-CORE, WORKER, DB-OLTP, DB-ANALYTICS, MONITORING, and MGT. The IDS sensor receives mirrored traffic rather than sitting inline, which decouples detection from forwarding: Suricata provides structured security evidence without adding forwarding latency or becoming an inline failure point, and enforcement remains under the fabric control path and the Zero-Trust management boundary.

A passive IDS alone cannot close the response loop. Even when a signature is reliable, a direct signature-to-block mapping can be unsafe because the same permitted service path may carry both legitimate traffic and malicious behavior: APP-CORE to DB-OLTP traffic may be allowed by the static policy, yet an abnormal transfer pattern on the same port may indicate compromise. BIZM therefore treats an alert as evidence to be interpreted with topology, segmentation policy, asset role, baseline behavior, active rules, and protected-asset constraints.

Table 3 gives the deny-by-default policy matrix of the evaluation environment. The normal service path runs from WEB to APP-GW to APP-CORE, which then reaches WORKER and the database tiers. DB-OLTP may replicate to DB-ANALYTICS, which is treated as a sink. MONITORING and MGT have broader reach because they support operations, but they are protected sources: isolating them automatically could disrupt the control or observability plane. The matrix defines both the static segmentation baseline and the policy context used by BIZM.

4.3. Verification-Guided Response Synthesis

The decision plane has two decision paths that share one validation and enforcement stage. Reasoning per alert is unnecessary when the response follows from the knowledge base and does not change between alerts, and it ties decision

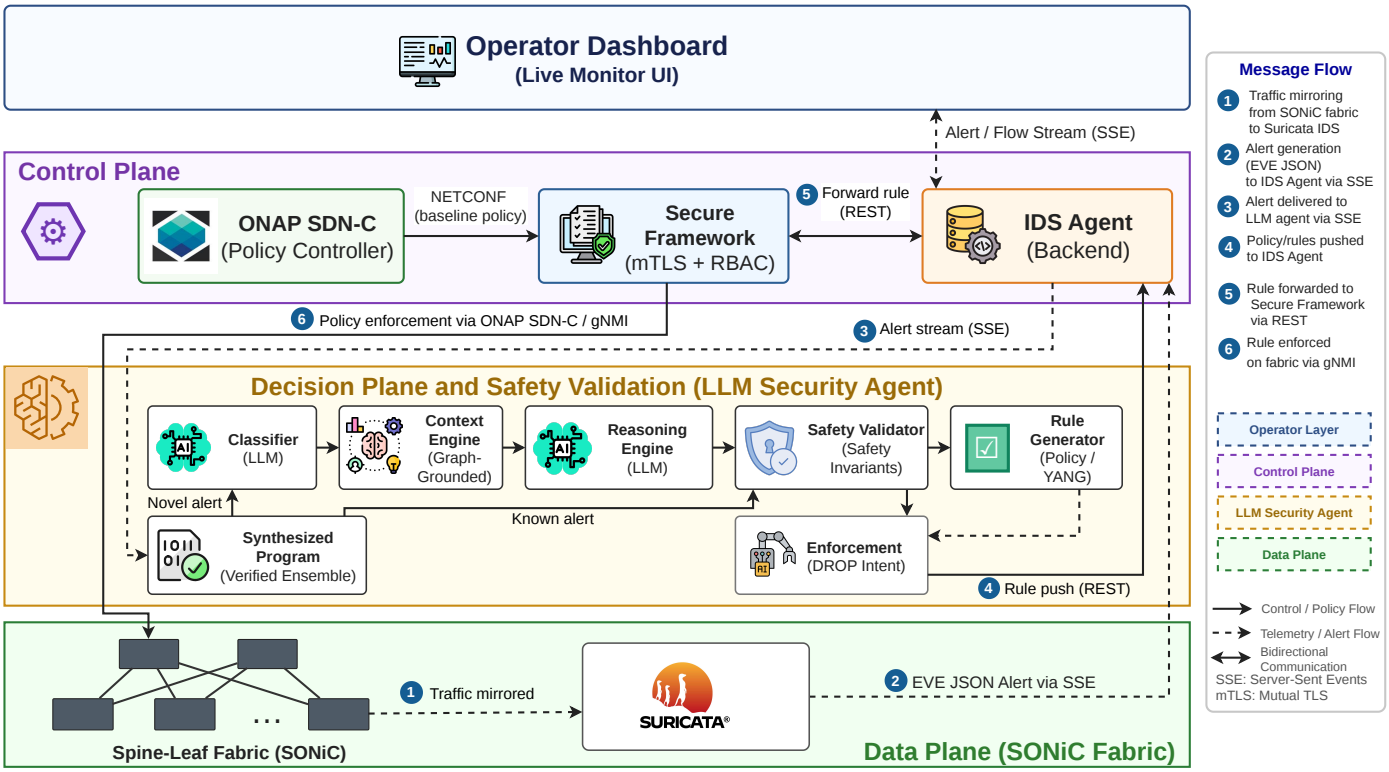


Figure 1: Overall BIZM architecture and bounded-isolation workflow. Mirrored east-west traffic from the SONiC fabric is inspected by Suricata, which generates EVE JSON alerts. The LLM Security Agent answers covered alerts from the synthesized, verified response program and performs classification, graph-grounded context retrieval, and LLM reasoning for the remaining alerts, followed by safety validation, enforcement planning, and rule generation. Accepted DROP intents are submitted to the IDS Agent, mediated by the Secure Framework and ONAP SDN-C, and enforced on the SONiC fabric through the southbound management path. Alerts and flow events are streamed to the operator dashboard through SSE.

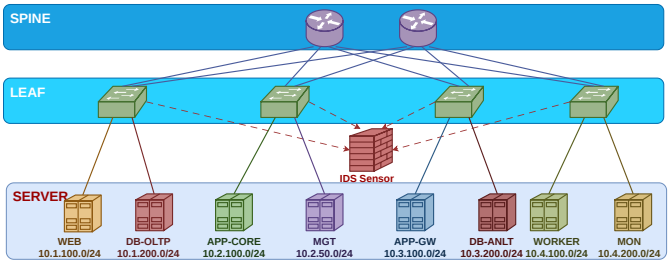


Figure 2: Eight-zone SONiC spine-leaf fabric with IDS mirroring. Workloads are attached to leaf switches and grouped into WEB, APP-GW, APP-CORE, WORKER, DB-OLTP, DB-ANALYTICS, MONITORING, and MGT zones. Dashed paths indicate mirrored traffic from the fabric to the Suricata sensor, which sits off the forwarding path.

latency, cost, and availability to the model provider (R8). The first path therefore uses the LLM ahead of time to synthesize the knowledge base into a program of $\mathcal{G}$ that answers covered alerts and abstains on the rest. The second path (Section 4.4) reasons over the alerts the program abstains on. Figure 3 and Algorithm 1 summarize the synthesis loop.

Program language. A program is an ordered list of rules over the eight features of Definition 1: the signature (cata-

Algorithm 1 Verification-guided response synthesis

Input: knowledge base K; language $\mathcal{G}$; verifier $\mathcal{V}_K$; ensemble size k; repair budget $\eta_{\max}$; sampling temperature T
Output: verified ensemble $\Pi^{\oplus}$ bound to $h(K)$, or $\perp$

$M \leftarrow \emptyset$
for $j = 1$ **to** k **do**
  $\Pi \leftarrow \textsc{Synthesize}(K, \mathcal{G}; T)$; $\eta \leftarrow 0$ ▷ one model call
  $C \leftarrow \mathcal{V}_K(\Pi)$ ▷ exhaustive over $D(K)$
  while $C \neq \emptyset$ **and** $\eta < \eta_{\max}$ **do**
    $\Pi \leftarrow \textsc{Repair}(\Pi, \textsc{Typed}(C))$; $\eta \leftarrow \eta + 1$
    $C \leftarrow \mathcal{V}_K(\Pi)$
  end while
  if $C \neq \emptyset$ **then** discard Π **and continue**
  $\Pi' \leftarrow \textsc{Review}(\Pi, \textsc{Summarize}(\Pi), K)$ ▷ self-review
  if $\mathcal{V}_K(\Pi') = \emptyset$ **then** $\Pi \leftarrow \Pi'$
  $M \leftarrow M \cup \{\Pi\}$
end for
if $M = \emptyset$ **then return** $\perp$ ▷ all alerts take the runtime path
$\Pi^{\oplus} \leftarrow \textsc{Majority}(M)$ ▷ Lemma 1
assert $\mathcal{V}_K(\Pi^{\oplus}) = \emptyset$
record $h(K)$, model, repair rounds, token cost
return $\Pi^{\oplus}$

Table 3: Eight-zone deny-by-default east-west policy matrix.

Src→Dst	WEB	GW	APP	WK	OLTP	ANL	MON	MGT
WEB		✓	✗	✗	✗	✗	✗	✗
APP-GW	✗		✓	✗	✗	✗	✗	✗
APP-CORE	✗	✗		✓	✓	✓	✗	✗
WORKER	✗	✗	✗		✓	✗	✗	✗
DB-OLTP	✗	✗	✗	✗		✓	✗	✗
DB-ANALYTICS	✗	✗	✗	✗	✗		✗	✗
MONITORING	✓	✓	✓	✓	✓	✓		✗
MGT	✓	✓	✓	✓	✓	✓	✓	

loged or unknown, from the alert), the severity (P1 to P4, from the alert), the source and destination zones (from the topology) with the derived policy verdict (allow, deny, intra-zone, or external, from the matrix), the protected-source flag (from the invariants), and the three history counts (prior alerts of the source, other targets of the source, and other ports on the target, from the alert history), with first-match semantics and the outcomes of Section 3.5. The scope of a DROP selects only the destination and port (flow, pair, or host); the language has no target field, so an enforced rule always blocks the alert source as a single host. Each DROP rule carries a rationale and the knowledge-base entries it relies on, which form its audit trace at runtime.

Exhaustive verification. For the deployed fabric, 29 signatures and the unknown value, four severities, nine zones including external, the protected flag, and seven consistent history combinations give $|D(K)| = 136{,}080$ points, of which $|R(K)| = 19{,}824$ can be emitted by the deployed IDS. The verifier evaluates the program on every point and returns every violation of I3, I4^s, I5, or C (Definition 3) as a typed counterexample naming the rule, the check, and an example input. Verifying one program takes about eight seconds on one core of the control-plane host, and verifying the deployed five-member ensemble about ten.

Synthesis and repair. The synthesizer gives the LLM the knowledge-base documents and the language definition and asks for one program that covers the whole fabric. The objectives are stated in priority order: obey the hard constraints, contain what the knowledge base identifies as requiring response, do not cut traffic it documents as legitimate, prefer the narrowest scope, and abstain when the knowledge base does not determine the answer. A rejected program is returned to the model with its typed counterexamples, and the model repairs it for at most $\eta_{\max}$ rounds; a program that still fails is discarded, which reduces the ensemble to the members that verified, and if no member verifies the synthesis returns no program and all alerts take the runtime path until the next attempt. This is the counterexample-guided loop of CEGIS [38] with an LLM as the inductive synthesizer and an exhaustive rather than symbolic verifier.

Self-review. A verified program is summarized per signature and policy verdict, and the model reviews this behavior

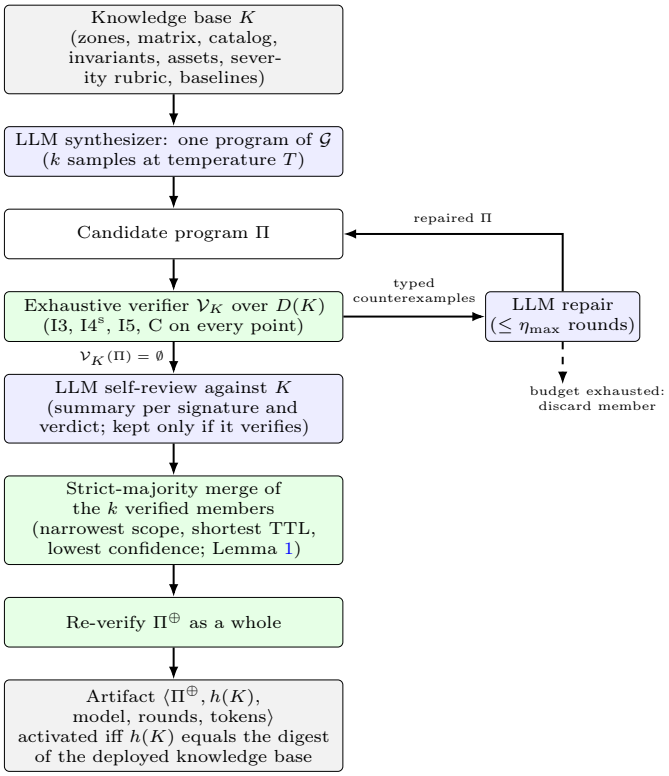


Figure 3: Verification-guided response synthesis (Algorithm 1). Shaded blue boxes are model calls; shaded green boxes are deterministic checks. A candidate program is verified exhaustively over the finite alert domain; typed counterexamples guide repair for at most $\eta_{\max}$ rounds; verified members are merged by strict majority, re-verified as a whole, and bound to the digest of the knowledge base they were synthesized from.

against the knowledge base. The review targets the second synthesis objective, agreement with the documented response, which the verifier does not check; the revision is kept only if it also verifies.

Ensemble. Because sampled programs differ, BIZM synthesizes k programs independently and deploys their point-wise strict majority as defined in Lemma 1: an action is taken only when a strict majority chooses it, otherwise the ensemble abstains, and among agreeing programs the narrowest scope, shortest TTL, and lowest confidence are kept. The ensemble is verified again as a whole.

Activation and runtime use. The synthesized artifact records the digest of the knowledge base it was synthesized from, the model, the repair rounds, and the token cost. The agent activates a program only if it was verified at synthesis time, its digest equals that of the running knowledge base, and it passes verification again in the agent process. A knowledge-base change therefore disables the old program until it is resynthesized, and all alerts take the runtime path in the meantime. At runtime, the features are extracted from the alert and from the source's alerts in the preceding ten minutes. A program decision becomes a bounded intent whose audit trace is

the rule rationale and its cited entries, and it passes the validators, rate limits, circuit breaker, and enforcement path of Section 4.4. Any error on the compiled path falls back to the runtime path rather than failing the alert.

4.4. Runtime Reasoning and Safety Validation

Alerts on which the program abstains, and all alerts while no verified program is active, take the runtime path. It is a structured pipeline of six functions: alert classification, context retrieval, LLM reasoning, safety validation, enforcement planning, and rule generation; the last three are shared with the compiled path. Contextual interpretation benefits from an LLM, whereas schema checking, target binding, TTL enforcement, protected-asset exclusion, blast-radius checking, and rule construction must remain deterministic.

Alert classification. Processing begins with normalization and deduplication. The Suricata EVE JSON alert is mapped to the response schema (source and destination addresses, protocol, destination port, signature, category, severity, timestamp), and duplicates within a suppression window are ignored. Alerts reach the classifier only when the program abstains. The classifier assigns a coarse response class: low-severity or duplicate events follow a log-only path, while high-priority alerts proceed to context retrieval and reasoning. This filter reduces unnecessary LLM calls and avoids enforcement attempts for events that are not eligible for automated containment.

Graph-grounded context. The context engine supplies the reasoning component with a constrained factual context rather than raw network state. The context graph includes zones, assets, leaf-switch attachment, IDS signatures, active DROP rules, protected address ranges, baseline communication edges, and prior incidents. For each alert it retrieves the source and destination zones, the policy-matrix entry, active rules, protected-asset status, recent similar incidents, and the expected impact of blocking the source. This gives the LLM the factual substrate needed to distinguish legitimate permitted-path behavior from suspicious behavior on the same allowed edge, and limits prompt exposure by supplying only the context needed for the current decision. The graph-query interface is read-only; any state-changing operation is outside the agent's tool set.

LLM reasoning. The reasoning engine evaluates the alert in context and produces a candidate bounded-isolation decision. The model is signature-informed but not signature-keyed: for known signatures the prompt includes trusted metadata such as severity, category, MITRE mapping, and expected disposition, but reasoning is reserved for the cases where static mapping is brittle, including permitted-path anomalies, unknown signatures, conflicting evidence, and scope selection under safety constraints. The output is schema-split: the enforcement-facing output is a compact structured action containing only the fields needed for validation and rule generation: the action (DROP or NONE), the source address, which must equal the alert source, the destination address, protocol, and destination port, a TTL in $[60, 3600]$ seconds, a confidence in $[0, 1]$, and the ATT&CK tactic and technique for audit, while the audit-facing output records the rationale, supporting evidence, alternatives, and follow-up recommendations. This split reduces parser failures and prevents a long explanation from delaying the enforcement path.

The runtime path keeps a decision cache for repeated alerts. An enforced decision is cached for 60 seconds under a key of signature, source, destination, port, protocol, and correlation signal. On a hit, the cached intent is rebound to the source of the current alert and passes validation again; alerts for a flow already covered by an active agent rule are suppressed until the rule expires. The cache saves a model call only within this window, whereas the compiled path removes it for every covered alert.

Fail-closed validation. The validator applies the invariants of Section 3.4. It checks schema conformance, sanitizes alert-derived text, verifies that the source exists in the context graph, rejects duplicate or conflicting rules, enforces the protected-asset set, rejects off-target isolation, clamps TTL to the configured interval, applies severity and confidence gates, and enforces per-source and global blast-radius limits. Any failed check results in non-enforcement and audit recording.

Prompt-injection handling has three layers. First, only the signature and category text are attacker-influenced; the prompt places them in a delimited untrusted-data block, while addresses, zones, port, signature identifier, and severity come from the parsed packet header and are marked as trusted. Second, the two text fields are scored against 15 patterns, including instruction overrides, role and chat-template tags, target coercion such as "block" followed by an address, and hidden Unicode characters; a field that matches two or more is redacted before it reaches the model. Third, and decisively, the enforcement target is bound to the header source after the model returns and checked against $\mathcal{N}$, so an instruction that passes the filter can change at most whether the source is contained, never which host is blocked. The compiled path does not read the alert text at all.

Enforcement planning and rule generation. Only an admissible decision is converted into a DROP intent. The enforcement planner binds the decision to the alert source, protocol, destination port, TTL, provenance metadata, and source-side enforcement scope, and the rule generator converts the intent into a policy/YANG-oriented request for the control plane. The model proposes the bounded action; deterministic components decide whether it is admissible and how it is represented for enforcement. Algorithm 2 summarizes the complete decision logic.

Algorithm 2 Fail-closed bounded-isolation decision

Input: alert $a = (s, d, \rho, \pi, \sigma, \ell, \kappa, \tau, t)$; policy P; baseline B; protected set $\mathcal{N}$; active rules $\mathcal{R}$; program $\Pi^{\oplus}$ (if active); threshold τ_c; TTL bounds $[\delta_{\min}, \delta_{\max}]$; blast-radius limits $\mathcal{L}$
Output: an authorized, provenance-marked bounded DROP rule, or a non-enforcing decision NONE/LOG_ONLY

$a' \leftarrow \textsc{NormalizeAndDeduplicate}(a)$
if a' is invalid or duplicated within the suppression window **then**
  $o \leftarrow$ NONE
else if $\ell \notin \{\text{P1}, \text{P2}\}$ **then**
  $o \leftarrow$ LOG_ONLY
else
  $u \leftarrow \Pi^{\oplus}(\phi_K(a', \textsc{History}(s)))$ ▷ compiled path
  if $\Pi^{\oplus}$ is inactive or $u.\alpha =$ ABSTAIN **then** ▷ runtime path
    $ctx \leftarrow \textsc{RetrieveContext}(a', P, B, \mathcal{R})$
    $ctx \leftarrow ctx \cup \textsc{Neighbors}(s) \cup \textsc{SimilarIncidents}(a') \cup \textsc{BlockImpact}(s)$
    if $\textsc{CacheHit}(a')$ **then**
      $u \leftarrow \textsc{CachedDecision}(a')$
    else
      $u \leftarrow \text{LLMDecision}(a', ctx)$
    end if
  end if
  if u fails schema validation **then**
    $o \leftarrow$ NONE
  else if $u.\alpha \neq$ DROP **then**
    $o \leftarrow$ LOG_ONLY
  else if $u.\theta \neq s$ or $u.\theta \in \mathcal{N}$ **then**
    $o \leftarrow$ NONE
  else if $\textsc{ConflictOrDuplicate}(u, \mathcal{R})$ **then**
    $o \leftarrow$ NONE
  else if $u.c < \tau_c$ **then**
    $o \leftarrow$ LOG_ONLY
  else if $\textsc{BlastRadiusExceeded}(u, \mathcal{L})$ or $\textsc{CircuitBreakerOpen}()$ **then**
    $o \leftarrow$ NONE
  else
    $u.\delta \leftarrow \textsc{Clamp}(u.\delta, \delta_{\min}, \delta_{\max})$
    $r \leftarrow \textsc{GenerateDropIntent}(u, \text{AGENT})$
    $o \leftarrow \textsc{AuthorizeAndEnforce}(r)$
  end if
end if
$\textsc{RecordAudit}(a', u, o)$

4.5. Zero-Trust Enforcement Plane

The enforcement plane prevents the LLM Security Agent from becoming an over-privileged network operator. The control path consists of the IDS Agent, the Secure Framework, and ONAP SDN-C. The IDS Agent integrates the decision layer with the control plane; the Secure Framework mediates policy requests using mTLS and RBAC; ONAP SDN-C acts as the policy controller that coordinates accepted operations and fabric-side enforcement.

Figure 4 details the enforcement boundary. The LLM Security Agent does not send device commands. An accepted DROP intent is submitted to the IDS Agent and forwarded to the Secure Framework through a restricted REST interface. The Secure Framework authenticates the request,

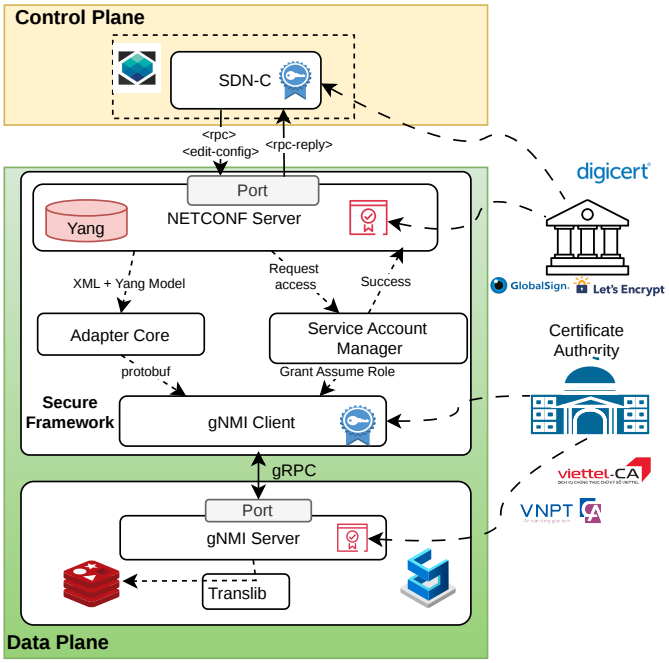


Figure 4: Identity-aware Secure Framework of the BIZM enforcement plane. The framework receives authorized bounded-response requests through a restricted northbound interface, validates operations against the YANG model, checks service-account permissions at command granularity under the restricted response role, translates accepted operations into gNMI/protobuf requests with provenance metadata, and pushes device-level rules to the data-plane switch through a mutually authenticated gRPC/gNMI channel. ACCEPT rules, deletions, unsupported operations, and requests without a valid identity fail closed at the authorization step.

binds it to a restricted service identity, checks command-level authorization, validates the operation against the YANG model, attaches provenance metadata, and translates the accepted intent into a gNMI-compatible operation, which is pushed to the data-plane switch through a mutually authenticated gRPC/gNMI channel.

The control-plane contract is intentionally narrow. BIZM can create temporary DROP rules, but it cannot create ACCEPT rules, delete unrelated rules, modify baseline segmentation policy, change routing state, or access protected management objects. Each accepted rule must be source-faithful, DROP-only, time-bounded, provenance-marked, and authorized under the restricted response role. Requests that target protected assets, request unsupported operations, lack valid credentials, or violate the allowed command set fail closed before reaching the fabric.

4.6. End-to-End Workflow

The end-to-end workflow follows the six steps of Section 4.1. It is closed-loop but not unrestricted: each iteration is bounded by action type, target fidelity, TTL, protected-asset exclusion, confidence threshold, severity threshold, blast-radius limit, identity-aware authorization, and audit recording. The response is reversible because

each rule has an explicit lifetime; when the TTL expires, the agent revokes the temporary rule, and the baseline segmentation policy remains unchanged.

Consider a compromised APP-CORE host that abuses its permitted database path to DB-OLTP. The static policy allows the zone pair, so the fabric does not block the flow by default. If Suricata raises a cataloged content signature, such as destructive SQL, the synthesized program issues a flow-scope DROP on the source without a model call. If the alert instead carries an unknown signature for a high-rate bulk transfer, the program abstains, and the runtime path retrieves the APP-CORE to DB-OLTP baseline, recent similar incidents, active rules, source leaf attachment, protected-asset status, and block-impact context. Because the edge is permitted, the response is not a fixed lookup: the reasoning engine compares the observed behavior with the baseline and proposes a bounded DROP on the source. In both cases, the validator confirms that the target equals the alert source, the source is not protected, the TTL is within bounds, and the confidence and severity gates are satisfied, and the rule generator emits a DROP intent that is authorized by the Secure Framework, coordinated through ONAP SDN-C, and enforced at the source-side leaf. A legitimate DB-OLTP replication burst toward DB-ANALYTICS, by contrast, matches the baseline and is recorded without enforcement.

This workflow captures the main design principle of BIZM: use the LLM to synthesize what the knowledge base determines and to reason where it does not, but constrain every consequential action through deterministic validation and Zero-Trust enforcement.

5. Implementation and Experimental Design

5.1. Testbed Implementation

We implement BIZM on a virtualized spine-leaf datacenter fabric emulated in GNS3 with SONiC switches [64]. Static microsegmentation is enforced as deny-by-default rules on the leaf switches, and runtime containment is installed as a temporary source-side DROP rule at the leaf nearest the suspicious workload. Each automated rule carries a TTL and provenance metadata, and the agent revokes it when the TTL expires.

Suricata monitors mirrored east-west traffic and exports EVE JSON alerts [11]. The IDS is placed off the forwarding path: it supplies the suspicious signal, while BIZM performs response-level decision making, following the common pattern in which lightweight sensing provides the initial evidence and deeper analysis runs outside the packet-forwarding path [34, 35, 12]. An IDS Agent subscribes to the alert stream, forwards normalized alerts to the decision plane, receives admissible DROP intents, and records live status for the operator dashboard.

Enforcement success is verified through four checks: the rule appears in the device rule datastore, the intended source can no longer reach the target service, unrelated traffic remains unaffected, and the installed rule carries the expected TTL and provenance mark. This distinguishes correct containment from accidental reachability loss or over-broad blocking. The verifier, synthesizer, and ensemble code of Algorithm 1 run on the control-plane host, a virtual machine with 32 vCPUs of an Intel Xeon (Cascade Lake) processor and 94 GB of memory.

5.2. BIZM Configuration

The runtime policy follows the eight-zone matrix of Table 3. The protected set $\mathcal{N}$ includes management hosts, monitoring hosts, IDS components, zone gateways, and control-plane components. Automatic enforcement is allowed only for P1 and P2 alerts. The runtime confidence threshold is $\tau_c = 0.70$: a decision at or above 0.85 is enforced, one between 0.70 and 0.85 is enforced with an operator notification, one between 0.50 and 0.70 is held for review, and a lower one is rejected. The compiled path requires $\tau_\Pi = 0.85$, and 0.90 for a DROP that overrides a permitted path. Algorithm 2 shows the single gate τ_c; the notification and hold bands are implemented inside AuthorizeAndEnforce and RecordAudit and do not change which decisions reach the fabric. Each DROP rule has a TTL in $[60, 3600]$ seconds. Blast-radius limits are five automated rules per minute, fifty active automated rules in total, and three rules per source address within five minutes. A circuit breaker pauses enforcement after three consecutive failed enforcement attempts.

The LLM is `gpt-oss-120b`, an open-weight mixture-of-experts model with about 117 billion parameters. The synthesizer uses it with `reasoning_effort` set to medium. Its input is the knowledge base of seven documents (zones, policy matrix, signature catalog, invariants, assets, severity rubric, and baselines), about 22K tokens. The deployed program is a strict-majority ensemble of $k = 5$ programs sampled at temperature 0.7, each repaired for at most $\eta_{\max} = 3$ rounds and reviewed once. Synthesis at temperature 0 is also reported as a single-program configuration. For the cross-model experiment (Section 6.3), the same configuration is run with `qwen-3.8-27b`, an open-weight model from a different family, which then serves as both the synthesizer and the runtime model.

On the runtime path, the same model serves both the reasoning and the classification roles, with `reasoning_effort` set to low. The reasoning call uses temperature 0.1, at most 3072 output tokens, and a 15-second timeout, and each P1/P2 alert receives a single reasoning call without self-consistency voting. The classification call uses temperature 0.0 and at most 1024 output tokens, and handles early severity/category filtering; duplicate alerts with the same source and signature are suppressed for 30 seconds. The reasoning prompt has four parts: trusted alert metadata, the untrusted alert text in its delimited block, the retrieved context with recent alerts from the same source, and the output schema. Prompts are fixed across runs, and no scenario-specific prompt tuning is performed. The enforcement-facing

output is constrained by the structured decision schema of Section 4.4; the reasoning trace is recorded for audit but is not used to authorize enforcement.

5.3. Research Questions, Scenarios, and Baselines

The evaluation addresses six questions.

- **RQ1 (Synthesis fidelity).** Does verification-guided synthesis produce a verified program that agrees with the knowledge base, and what does the verifier catch that the synthesizer alone would have deployed?
- **RQ2 (Robustness to change).** Does resynthesis keep the program correct when the knowledge base changes?
- **RQ3 (Sensitivity).** How do ensemble size and model choice affect coverage, agreement, and synthesis cost?
- **RQ4 (Live containment).** Does BIZM contain functional east-west threats on the live testbed, at what latency, and at what cost to legitimate traffic?
- **RQ5 (Adversarial safety).** Does the safety mechanism prevent unsafe enforcement under alert manipulation and under a coordinated multi-source campaign?
- **RQ6 (Residual reasoning and scalability).** How does the runtime path handle alerts the program does not cover, what does each path cost, and how does verification scale with the fabric?

RQ1 to RQ3 are evaluated offline over the alert domain. For RQ2, four modified copies of the knowledge base represent changes an operations team makes between syntheses: six new signatures, two of which detect abuse inside permitted connections (V1); a ninth crown-jewel zone, PAYMENTS, with its matrix rows, gateway, and two zone-pair signatures (V2); three documented responses changed in prose only (V3); and all three changes together (V4). Each variant is resynthesized, while the baselines are left as written. For RQ3, the ensemble size is varied over $k \in \{1, 3, 5, 7, 9\}$ with 20 ensembles per k drawn without replacement from the pool of 30 syntheses, and the synthesis configuration is repeated with a second open-weight model.

Table 4 summarizes the testbed scenarios. Five scenarios use live east-west traffic and exercise the full detection-to-enforcement path. Five scenarios use controlled injected alerts to stress the decision logic, protected-asset handling, prompt-injection resistance, policy-matrix coverage, unknown signatures, and multi-stage correlation. Live scenarios are repeated ten times. Injected scenarios are repeated three times with the synthesized program, whose decisions are deterministic, and ten times in the runtime-only configuration. These repetitions measure run-to-run stability and should not be interpreted as a large independent traffic corpus. A matrix sweep injects 75 alerts generated mechanically from every off-diagonal cell of the policy matrix, the signature catalog, and the protected set, five times each. Scenario S11 launches a coordinated campaign from $n_s \in \{3, 5, 8, 12\}$ non-protected WEB sources, each violating the policy toward DB-OLTP and raising its alert again every 10 seconds until it is contained, as an attacking host keeps triggering the IDS; it is repeated five times per n_s. In the decoy variant, twelve spoofed non-protected addresses start the campaign and one real source starts 5 seconds later, and a further configuration raises the per-minute limit to $L' = 12$ for $n_s = 12$.

Table 4: Evaluation scenarios and repetitions (N: BIZM/runtime-only).

ID	Description	Type	N
S1	WEB to DB-OLTP direct, application-chain bypass	Denied-cell violation	10/10
S2	APP-CORE reverse connection to WEB	Denied-cell lateral movement	10/10
S3	DB-OLTP outbound beacon from sink tier	Denied-cell behavioral violation	10/10
S4	Destructive SQL on permitted APP-CORE to DB-OLTP edge	Permitted-path anomaly	10/10
S5	Management and monitoring baseline traffic	Legitimate FP control	10/10
S6	Denied-cell injections and legitimate flows	Matrix coverage and FP	3/10
S7	Prompt injection, severity bypass, spoofed-source alerts	Adversarial safety	3/10
S8	Protected management or monitoring host acting maliciously	Never-block invariant	3/10
S9	Known, unknown, and legitimate signatures	Reasoning ablation	3/10
S10	Three-stage campaign from a single source	Kill-chain correlation	3/10
S11	Coordinated campaign from n_s sources (direct and spoofed-decoy variants)	Blast-radius exhaustion	5/n/a

We compare BIZM against three baselines and one ablation.

- *SOAR-A* is a signature-keyed responder that issues a DROP against the alert source for a fixed set of known-dangerous signatures and takes no action otherwise; a host-isolation variant is used in the matrix sweep. It represents fixed playbooks.
- *Baseline-B* uses the same LLM decision model as BIZM but removes deterministic safety validation and least-privilege enforcement. It receives raw alert text and emits an action directly, which isolates the contribution of the safety controls.
- *Baseline-C* is a deterministic contextual rule written by the authors over the same topology, policy matrix, and protected set: it contains a severe alert on a denied zone pair and, on a permitted pair, only for a hand-written list of abuse signatures. It is evaluated as deployed, with zone-level protection of MGT and MONITORING, and in a strongest variant that protects exactly the hosts of the protected set and reads zones and the matrix from

the current knowledge base. It separates the contribution of the LLM from that of the context itself.

- *Runtime-only* is BIZM with the synthesized program disabled, so every alert is reasoned by the LLM. This ablation isolates the compiled path.

5.4. Evaluation Metrics

For response correctness, each labeled decision is classified as TP, FP, TN, or FN. A positive case is a true threat whose correct response is a bounded DROP decision [35]; a negative case is benign, ambiguous, low-severity, or unsafe to enforce and should not trigger automated containment. We report precision, threat-containment rate (TCR), F1-score, false positive rate (FPR), and accuracy:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{TCR} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{TCR}}{\text{Precision} + \text{TCR}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

TCR is a response-level metric: it measures the fraction of true threat cases for which the system issues an admissible bounded DROP decision, and it should not be interpreted as an IDS detection rate, because BIZM operates after an alert has been generated.

For RQ1 to RQ3, ground truth is derived mechanically from the knowledge base, never from a program. For an alert with a cataloged signature, the documented response governs. It is read from the *Response* line of the signature's prose entry in the catalog, which has no structured counterpart, so a system that reads only the structured fields, such as Baseline-C, cannot reproduce it. A documented DROP means contain, a documented log-only means do not contain, and a DROP conditioned on correlation means contain only after a prior alert from the source. Protected or external sources and P3 or P4 alerts are never contained. Points the knowledge base does not settle are left ungraded: a correlation-conditioned signature with no prior alert, and a log-only signature of severity P1 or P2 on a denied pair, where the pair itself is a policy violation. Unknown signatures are reported separately. On the graded points we report *accuracy* over the points a system decides and *coverage*, the fraction it decides rather than abstains on. Because these results enumerate the domain rather than sample it, they carry no sampling interval. For the synthesis loop we additionally report the verification outcome of each candidate program, the number and invariant type of counterexamples per round, the repair rounds used, whether the self-review revision was accepted, and the agreement and coverage of the *unverified* candidates, that is, of what the synthesizer would have deployed without the verifier.

We also report the following metrics.

- **Latency.** Detection latency from traffic onset to the IDS alert; decision latency from alert ingestion to a decision accepted by the Secure Framework; enforcement latency from that decision to rule visibility in the leaf datastore. Medians, 95th percentiles, and empirical distributions are reported.
- **Exposure.** For the matrix sweep, the service edges a rule closes and the legitimate baseline flows it cuts.
- **Safety.** Safety-violation rate, protected-service availability, never-block respect for protected assets, and, for the multi-source campaign, the fraction of sources contained before and after blast-radius saturation.
- **Reasoning.** Correct-decision rates on known and unknown signatures.
- **Overhead.** Median and tail decision latency, model tokens per decision, the one-time cost of synthesis, verifier time as a function of $|D(K)|$, and sustained compiled-path throughput.

Proportions are reported with exact Clopper-Pearson 95% intervals, and paired comparisons with the exact McNemar test.

Table 5: Agreement with the knowledge base on the deployed snapshot V0 and after the four changes V1 to V4 (BIZM resynthesized, Baseline-C unchanged). Graded: graded points of the snapshot.

		Baseline-C		BIZM	
Knowledge base	**Graded**	**Depl.**	**Strong.**	**Acc.**	**Cov.**
V0, single (T=0)	14,970	94.54%	96.07%	100.00%	97.5%
V0, ensemble (k=5)	14,970	94.54%	96.07%	100.00%	100.0%
V1 new signatures	19,520	93.73%	94.98%	100.00%	95.0%
V2 new zone	18,705	93.03%	96.26%	100.00%	97.5%
V3 prose retuning	15,034	94.46%	96.09%	100.00%	100.0%
V4 all changes	24,428	92.03%	95.24%	99.97%	92.3%

6. Results, Evaluation, and Discussion

6.1. Synthesis Fidelity: Agreement with the Knowledge Base (RQ1)

Table 5 compares synthesized programs with Baseline-C over the deployed knowledge base (V0) and, for RQ2, over its four variants. Of the 19,824 realizable points of V0, 14,970 are graded, 822 are ungraded, and 4,032 carry an unknown signature. Every deployed program passed verification with no violating point. The single program synthesized at temperature 0 decides 97.5% of the graded points and agrees with the knowledge base on all of them, and it was identical across five syntheses. The ensemble decides every graded point and agrees on all of them. Its members varied individually, with coverage between 69.0% and 100% and one member missing 966 contain points, and the strict-majority vote removed every member error, as Lemma 1 guarantees for safety and as the data show for agreement.

Baseline-C errs in both directions. Its hand-written list of permitted-path abuse signatures includes three signatures that the catalog documents as log-only, so even its strongest variant blocks 588 points that the knowledge base says to log (no false negatives, 588 false positives).

The deployed variant also protects every host of MGT and MONITORING instead of the hosts in the protected set and misses 544 contain points (544 false negatives, 273 false positives), and it enforces on 518 points with an unknown signature, which BIZM defers to the runtime path. The rule is consistent with the policy matrix but not with the prose of the catalog, which it never reads. The synthesized program is derived from that prose, and this is where the LLM contributes. Synthesizing the ensemble took 117 seconds and 341K tokens. A mechanically compiled program that reads only the structured fields of the catalog, without the LLM, agrees with the knowledge base on 93.55% of the graded points when it must decide all of them, missing 966 contain points on permitted and outbound pairs, or agrees on all of the 89.6% it can decide when it abstains elsewhere; on V1 to V4 the same program reaches 93.0% to 93.9%, or 100% at 88.0% to 90.2% coverage. This bounds the contribution of the LLM to the roughly 10% of graded points whose response is stated only in the prose of the catalog.

What the verifier catches. The upper part of Table 6 reports the dynamics of Algorithm 1 for the six syntheses of the deployed knowledge base (five ensemble members and the temperature-0 program). Of the six initially synthesized programs, three failed verification at round 0 with 24 counterexamples in total (15, 3, and 6), covering 18,648 violating inputs; one repair round sufficed for each of them, and no candidate was discarded. The self-review revision verified and was kept for all six programs; it raised the coverage of the realizable points for three ensemble members, from 91.0% to 92.1%, from 88.7% to 95.5%, and from 85.9% to 90.9%. The round-0 candidates of these syntheses were not retained, so the following figures come from a pool of 30 further syntheses of V0 at $T = 0.7$ in which every candidate was kept. Eight of the 30 failed verification at round 0 with 47 counterexamples, 41 of them on the TTL and scope check $I4^s$, four on I5, and two on I3; every one was repaired within two rounds (0.33 on average), and none was discarded. The eight unverified round-0 candidates agreed with the knowledge base on 98.75% of the graded points they decided, against 100% for the 22 that verified at once, and one of them would have issued 448 unsafe DROP decisions on protected sources, which quantifies what verification-guided repair removes before deployment. Self-review changed the decision on 257 to 3,736 graded points (median 680) in 15 of the 30 syntheses.

Synthesis ablation. The lower part of Table 6 isolates the contribution of each stage of the loop over the pool of 30 independent syntheses of V0 at temperature 0.7, read at four stages: one-shot synthesis with no verifier feedback, repair only, repair with self-review, and the full ensemble of five. One-shot candidates verify in only 73.3% of the cases, agree with the knowledge base on 99.67% of the graded points they decide (95% CI over syntheses [99.17, 100]), and include 448 unsafe DROP decisions; repair raises the verification rate to 100% (exact McNemar $p = 0.008$ on the eight discordant syntheses) and removes every unsafe decision while leaving agreement on decided points essentially unchanged at 99.77%; self-review raises agreement to 100.00% and coverage from 93.6% to 96.7%; and ensembles of five raise coverage to 99.6% (95% CI [99.0, 100]) at 100% agreement. The same ablation on a pool of 20 syntheses of V4 gives 70% verified at round 0 (one draft could not be parsed), no unsafe decision at any stage, agreement of 99.80%, 99.64%, and 99.68% for the three single-program stages, and 100.00% agreement at 98.9% coverage for ensembles of five. This is the direct evidence for the title claim: the verifier, not the synthesizer, is what makes the deployed program safe, and the ensemble, not any single sample, is what makes it nearly complete.

Table 6: Synthesis-loop dynamics for the six deployed syntheses (upper part) and synthesis ablation over 30 syntheses at $T = 0.7$ (lower part).

Candidate	CE_0/Ver.	Rnds	Acc.	Cov.	Unsafe	Tok.
Deployed syntheses (round-0 candidate before repair)						
T=0 single	15	1	n/a	n/a	n/a	96K
Member 1	0	0	n/a	n/a	0	54K
Member 2	3	1	n/a	n/a	n/a	93K
Member 3	0	0	n/a	n/a	0	54K
Member 4	6	1	n/a	n/a	n/a	80K
Member 5	0	0	n/a	n/a	0	60K
Ablation arms (output the arm would deploy; mean, 95% CI)						
One-shot (no verifier)	73.3%	0	99.67%	94.3%	448	26K
Repair only	100%	0.33	99.77%	93.6%	0	36K
Repair + review	100%	0.33	100.00%	96.7%	0	66K
Ensemble (k=5)	100%	0.26	100.00%	99.6%	0	322K

Upper part: CE_0 is the number of counterexamples at round 0 (breakdown by check in the text), Rnds the repair rounds used, and Acc./Cov./Unsafe the agreement, coverage, and unsafe DROP decisions (protected or external source) of the *unverified* round-0 candidate on the graded points, which were not retained for these syntheses (n/a). Lower part: Ver. is the fraction of arm outputs with $\mathcal{V}_K(\Pi) = \emptyset$ and Rnds the mean repair rounds; Acc./Cov./Unsafe refer to what the arm would deploy, as means over the 30 syntheses (20 ensembles of five drawn from them).

6.2. Resynthesis under Knowledge-Base Change (RQ2)

Rows V1 to V4 of Table 5 report the four knowledge-base changes. Resynthesis keeps agreement at 100% on V1 to V3 and at 99.97% on V4. The seven V4 errors concern SID 9000012 on PAYMENTS to DB-OLTP: the variant permits that pair but leaves the signature's detection logic unchanged, so the knowledge base contradicts itself there. Baseline-C, which is not rewritten, misses the two new abuse signatures of V1 and keeps blocking SID 9000050 after V3 documents it as log-only. Its deployed variant cannot place the PAYMENTS zone and misses 1,009 contain points in V2. The strongest variant follows the new zone and matrix rows, which are structured, but not changes stated in prose. The ensemble coverage falls to 92.3% on V4 because its members disagree on some new signatures, and those points go to the runtime path rather than to a wrong decision. Each resynthesis took under four minutes, whereas the rule requires a code change and redeployment.

The verifier rejected 8 of the 20 initial ensemble candidates across the four variants (2, 2, 1, and 3 for V1 to V4), each of which was repaired within two rounds, and no candidate was discarded; the four temperature-0 programs verified at round 0. Resynthesis with `qwen-3.8-27b` (Section 6.3) keeps agreement at 97.94%, 99.37%, 100%, and 99.97% on V1 to V4, with coverage between 96.7% and 98.6%; all of its errors leave a point uncontained, and none blocks a permitted flow.

6.3. Sensitivity to Ensemble Size and Model Choice (RQ3)

Ensemble size. The upper part of Table 7 varies k. By Lemma 1 every ensemble is verified; what changes is the coverage-abstention trade-off and the synthesis cost. Coverage rises from 96.1% at $k = 1$ to 99.6% at $k = 5$, peaks at 99.9% for $k = 7$, and falls to 99.5% at $k = 9$, where a strict majority is harder to reach; agreement on decided points remains 100% for every k, and the one-time token cost grows linearly in k. At $k = 5$, 88% of the abstained points carry one of the three permitted-path signatures that the catalog documents as log-only (SIDs 9000051, 9000052, and 9000061), mostly for sources with earlier alerts, where members disagree whether correlation turns logging into containment. These are exactly the points on which the knowledge base is least explicit, and the same three signatures that Baseline-C blocks.

Model choice. The lower part of Table 7 repeats the synthesis with a second open-weight model, `qwen-3.8-27b`, under the same prompts, budget, and verifier. None of its five round-0 drafts was admitted: four could not be parsed as a program and the fifth was rejected by the verifier. Every candidate reached verification within two repair rounds (1.2 on average), and none was discarded. Over a pool of 30 syntheses, 23 failed at round 0 (16 unparseable, 7 rejected with 11 counterexamples, 9 of them I4), all 30 were verified within $\eta_{\max}$ rounds, and the 20 ensembles of $k = 5$ drawn from the pool reach 99.18% agreement at 97.1% coverage, against 100% and 99.6% for `gpt-oss-120b`, at 760K against 322K tokens. On V4, all 20 of its round-0 drafts failed (18 unparseable, 2 rejected), all 20 were verified after repair, and its $k = 5$ ensembles reach 99.47% agreement at 98.0% coverage, against 100% and 98.9% for `gpt-oss-120b`. The deployed ensemble agrees with the knowledge base on 99.17% of the points it decides, with coverage 96.0%; all 120 errors leave uncontained a point that the knowledge base contains, and none blocks a flow that the knowledge base permits. Synthesis cost 802K tokens, 2.4 times that of `gpt-oss-120b`. Across both models, the verifier never admitted a program with an unsafe decision, so model choice affects coverage, agreement, and cost but not safety, which is the property the architecture is designed to make model-independent (R6).

Full system on the second model. To test whether the testbed results depend on the model, the full evaluation is repeated with `qwen-3.8-27b` as both the synthesizer and the runtime model: the live scenarios S1 to S5 (ten runs per configuration), the injected scenarios S6 to S10, the matrix sweep (75 cases, one repetition), and S11. Table 8 compares the two models. All outcomes that the architecture is designed to guarantee are unchanged: every threat run is contained at the correct source, S5 produces no enforcement, no S7 case violates an invariant, the never-block invariant holds in every S8 case, and S11 reproduces the containment pattern of the default limits row for row (five sources in the first minute, all twelve within 140 to 150 seconds), because the campaign is decided by the program and bounded by the limiter. Model choice changes the runtime path. The runtime-only configuration with `qwen-3.8-27b` decides in 6.38 seconds at the median against 3.12 seconds, and in the BIZM configuration the S2 runs, which reach the runtime path, raise the 95th percentile to 6.96 seconds, while the program decides the other threat runs in 0.05 seconds. The program makes no model call, so this median differs from the 0.59 seconds of the `gpt-oss-120b` campaign only through enforcement: the leaf-dependent acknowledgment delay of Section 6.4 was absent in this later campaign, and program decisions took 0.04 to 0.08 seconds on every leaf. On decisions, the two models differ in single cases: `qwen-3.8-27b` answers all S9 cases correctly in both configurations, and once leaves an S6 denied-cell injection as log-only in the runtime-only configuration, as `gpt-oss-120b` does once. Baseline-B with `qwen-3.8-27b` violates an invariant in 60 of 90 S7 decisions (66.7%, against 100%) and still blocks every protected S8 host, so the unguarded agent remains unsafe with either model.

Table 7: Sensitivity of synthesis to ensemble size (upper part, `gpt-oss-120b`, 20 ensembles per k from 30 syntheses) and to model choice (lower part, $k = 5$).

Setting	Pass@0	Rounds	Acc.	Cov.	Tokens
Ensemble size k					
$k = 1$	75%	0.35	100.00%	96.1%	67K
$k = 3$	70%	0.33	100.00%	99.2%	196K
$k = 5$	78%	0.26	100.00%	99.6%	322K
$k = 7$	72%	0.38	100.00%	99.9%	476K
$k = 9$	73%	0.34	100.00%	99.5%	591K
Synthesizer model					
`gpt-oss-120b`	60%	0.4	100.00%	100.0%	341K
`qwen-3.8-27b`	0%	1.2	99.17%	96.0%	802K

Pass@0: fraction of candidates verified at round 0; Rounds: mean repair rounds among verified candidates; Acc./Cov. on the 14,970 graded points; upper part: means over the 20 ensembles; lower part: the deployed ensemble.

6.4. Closed-Loop Containment and Latency (RQ4)

Table 9 reports the per-scenario outcomes of the live scenarios and of the injected S6 decisions, together with the median latencies of the live runs; Decide and Visible are measured from the alert to an accepted decision and to rule visibility in the leaf datastore. In the four threat scenarios, BIZM installs the correct source-side DROP rule in all ten runs, each decided by the synthesized program, including S4, a permitted-path anomaly on the APP-CORE to DB-OLTP edge. S4 is flagged by the destructive-SQL content

Table 8: Full-system results with each model (BIZM and runtime-only configurations).

	BIZM		Runtime-only	
Measure	**gpt-oss**	**Qwen**	**gpt-oss**	**Qwen**
S1 to S4 contained	40/40	40/40	40/40	40/40
S5 without enforcement	10/10	10/10	10/10	10/10
Decide, median (s)	0.59	0.05	3.12	6.38
Decide, 95th pct. (s)	2.70	6.96	4.51	8.60
S6 (TP/FN/FP/TN)	15/0/0/12	15/0/0/12	49/1/0/40	49/1/0/40
S7 violations	0	0	0	0
S8 never-block respected	100%	100%	100%	100%
S9 correct	25/27	27/27	88/90	90/90
Sweep accuracy	100.0%	100.0%	100.0%	100.0%
Sweep edges closed / cut	7/7	7/7	7/7	7/7

gpt-oss: `gpt-oss-120b`; Qwen: `qwen-3.8-27b`. Decide over the 40 enforced live runs. Injected scenarios: 3 (BIZM) and 10 (runtime-only) repetitions; sweep: 5 repetitions with `gpt-oss-120b`, 1 with `qwen-3.8-27b`.

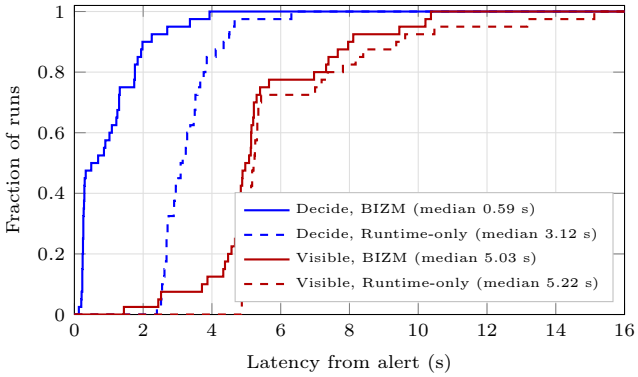


Figure 5: Empirical distributions of decision latency and rule-visibility latency on the live scenarios (S1 to S4, $N = 40$ enforced runs per configuration).

signature, which B lists as abuse of that edge, so the documented response applies although the zone pair is permitted. The false-positive control S5 produces no automated enforcement in all ten runs. All forty enforced runs pass the post-enforcement checks: rule presence, source-side drop behavior, blocked reachability from the offending source, unaffected unrelated traffic, and TTL within bounds.

The compiled path reduces the median decision latency from 3.12 to 0.59 seconds, and the 95th percentile from 4.51 to 2.70 seconds. Program evaluation itself takes 2 to 17 ms (Section 6.8); the remainder of the compiled-path decision latency is the acceptance round trip through the IDS Agent and the Secure Framework, which the decision waits for because rule installation is synchronous. This round trip also explains the difference between S1 and S3 (1.27 and 1.10 s) and S2 and S4 (0.29 and 0.26 s): the Secure Framework installs an agent rule on the leaf that hosts the source zone, and the acknowledgment took about one second on the leaf of WEB and DB-OLTP and about 0.3 seconds on the leaf of APP-CORE. The time until the rule is observed in the leaf datastore is similar on both paths, with medians of 5.03 and 5.22 seconds. It is an upper bound set by the observation rather than by the decision: the harness looks for the decision every 10 seconds, starting after the 5-second arming step, and then checks the leaf datastore, so any decision taken within about five seconds of the alert is observed at the same poll on both paths. The runtime-only S3 visibility median of 7.59 s has the same cause: the harness starts looking only after the arming step, and in six of the ten runs the beacon alert and its decision preceded the end of that step, so the decision-to-visibility interval was 3.8 to 9.9 seconds in those runs against 2.1 to 2.3 seconds in the other four. Figure 5 shows the empirical distributions over all forty enforced runs of each configuration. The compiled-path decision latency stays below 1.5 seconds in 30 of the 40 runs and below 4 seconds in all of them, whereas the runtime-only distribution starts at 2.4 seconds, the time of a model call, and reaches 6.3 seconds; the visibility distributions overlap. Once the decision is synthesized, the closed loop is bounded by the sensor and the enforcement path. This latency is not appropriate for packet-level mitigation, but it is suitable for containment of lateral movement and substantially faster than manual triage.

Over the 77 labeled live and S6 decisions, BIZM makes no error: precision 100% (95% CI [93.5, 100]), threat-containment rate 100% [93.5, 100], false positive rate 0% [0.0, 15.4], and accuracy 100% [95.3, 100]. The runtime-only configuration, over 140 decisions, reaches precision 100% [95.9, 100], TCR 98.9% [94.0, 100] (F1 99.4%), FPR 0% [0.0, 7.1], and accuracy 99.3% [96.1, 100]. Its single miss is an inbound-management case answered log-only: a DB-OLTP host opening SSH toward MGT (SID 9000011). The compiled path contains this case in all three runs, because it decides from the documented response and the denied verdict of the pair; this is the policy-verdict confirmation that the runtime path lacks. In four of its forty legitimate S6 decisions the runtime LLM proposed a DROP, which the policy-conflict check (three) and the severity gate (one) rejected before enforcement. The intervals show that these figures are evidence of feasibility on this scenario set, not a guarantee of zero errors.

The upper part of Table 10 reports the matrix sweep. Static segmentation answers none of the 42 containment cases. Host isolation contains the most but cuts 6.3 times more legitimate flows than flow-scope containment, and it over-blocks the four protected-source cases. The synthesized program decides all 75 cases at flow scope and agrees with the runtime path and Baseline-C, which coincide with it on these single-alert cases; their differences lie in the parts of the domain the sweep does not sample (Section 6.1). Every service edge removed falls on a permitted pair, because on a denied pair the static matrix has already removed it. Because the compiled path is deterministic, the five runs of a case are not independent; the table therefore reports intervals over the 75 distinct cases, and the exact McNemar test between host isolation and BIZM on those cases gives $p = 0.125$ (four discordant cases), whereas treating the 375 decisions as the unit would give $p = 1.9 \times 10^{-6}$. The sweep is a coverage check of the policy matrix, not a sample from

Table 9: Per-scenario decision counts (TP/FN/FP/TN) and median latency in seconds for the live scenarios S1 to S5 and the injected scenario S6. Detect: traffic onset to IDS alert; Decide: alert to accepted decision; Visible: alert to rule visibility in the leaf datastore.

		Decisions (TP/FN/FP/TN)			BIZM		Runtime-only	
Scn.	**Type**	**BIZM**	**Runtime-only**	**Detect**	**Decide**	**Visible**	**Decide**	**Visible**
S1	Threat, live	10/0/0/0	10/0/0/0	3.30	1.27	5.21	3.02	5.14
S2	Threat, live	10/0/0/0	10/0/0/0	3.95	0.29	4.89	3.59	5.15
S3	Threat, live	10/0/0/0	10/0/0/0	2.90	1.10	5.53	2.96	7.59
S4	Threat, live	10/0/0/0	10/0/0/0	3.45	0.26	4.76	2.91	5.21
S5	Legitimate, live	0/0/0/10	0/0/0/10	n/a	n/a	n/a	n/a	n/a
S6	Mixed, injected	15/0/0/12	49/1/0/40	n/a	n/a	n/a	n/a	n/a
Total		55/0/0/22	89/1/0/50					

Table 10: Comparison with the baselines on the matrix sweep (upper part; 75 distinct cases, each repeated 5 times with identical outcomes; SOAR-A uses host isolation) and on the S9 reasoning ablation (lower part; correct decisions). Edges and Cut are totals over the 75 distinct cases.

Measure	**Static**	**SOAR-A**	**Basel.-C**	**RT-only**	**BIZM**
Sweep accuracy	44.0%	94.7%	100.0%	100.0%	100.0%
Edges closed	0	88	7	7	7
Legit. flows cut	0	44	7	7	7
S9 malicious, known	n/a	30/30	n/a	30/30	9/9
S9 malicious, unknown	n/a	0/30	n/a	28/30	7/9
S9 legit., unknown	n/a	30/30	n/a	30/30	9/9
S9 total	n/a	60/90	n/a	88/90	25/27

Sweep accuracy over the 75 distinct cases, 95% Clopper-Pearson intervals: Static [32.5, 55.9]; SOAR-A [86.9, 98.5]; Baseline-C, Runtime-only, and BIZM [95.2, 100]. Basel.-C: Baseline-C; RT-only: Runtime-only.

traffic, and its intervals should be read as such.

6.5. Adversarial Safety and Bounded-Isolation Guarantees (RQ5)

The upper part of Table 11 reports adversarial-safety results. Scenario S7 contains prompt-injection alerts that attempt to redirect isolation to protected control-plane addresses, low-severity alerts that attempt to bypass the severity gate, and spoofed-source cases. BIZM produces no unsafe enforcement in its 27 decisions, nor in the 90 decisions of the runtime-only configuration, which bounds the violation rate on this scenario set below 12.8% and 4.0% at 95% confidence. With the synthesized program, the injected alerts never reach a model: the program decides from the alert source, zones, and signature and ignores alert text, so the injected instructions have nothing to act on. The injection cases were contained at their true source, and the spoofed-source and low-severity cases were removed by the protected-asset and severity gates. In the runtime-only configuration, the protected-asset set and off-target checks reject injected targets.

Baseline-B, which uses the same LLM without deterministic safety layers, violates safety in 100% of its 90 adversarial decisions (95% lower bound 96.0%). It follows injected instructions, targets protected services, or accepts unsafe redirections. Scenario S8 stresses the protected-asset invariant directly by making management or monitoring hosts appear malicious. BIZM respects the never-block constraint in all 12 protected-host decisions and the runtime-only configuration in all 40, with 95% lower bounds of 73.5% and 91.2%; both surface the events for review, whereas Baseline-B isolates protected hosts. BIZM's safety is therefore not a property of the model; it comes from deterministic validation and least-privilege enforcement around the model.

The adversarial cases are author-written, but on the compiled path two attack classes are excluded for every alert, not only the tested ones. Target-fidelity confusion cannot occur because the language has no target field, and TTL abuse cannot occur because the verifier checks the TTL bounds on every point of $D(K)$ (Proposition 2). The spoofed-source cases of S7 spoof protected addresses, which the protected-asset gate removes. Spoofing a *non-protected* address is not a safety violation under the invariants: the rule that results is bounded, time-limited, and scoped to connections the spoofed address initiates toward one destination and port. It is an availability vector, and the decoy variant of S11 below measures it.

Coordinated multi-source campaign. A campaign from many sources attacks a different control: the global blast-radius limit. Scenario S11 launches policy violations from n_s non-protected sources within 60 seconds, in a direct variant and in a decoy variant in which the alerts are produced by spoofed non-protected addresses whose purpose is to exhaust the limits. The lower part of Table 11 shows that with the default limits (five rules per minute, fifty active) BIZM contains all sources for $n_s \leq 5$ within 1.9 seconds, and the first five within the first minute for $n_s \in \{8, 12\}$, after which the per-minute limit holds further containment until the next window. Because an uncontained source raises its alert again and the 30-second alert dedup window applies, the remaining sources are contained at the next windows: all eight within 70 to 81 seconds and all twelve within 140 to 162 seconds. Every decision was taken by the synthesized program, all 265 enforced rules were present on the fabric, no address outside the campaign was blocked in any run, and the 230 deferred decisions were all rejected by the per-minute limit.

Table 11: Adversarial safety on S7 and S8 (upper part) and the multi-source campaign S11 (lower part, five runs per row; default limits unless L' is given).

Metric	**BIZM**	**RT-only**	**Baseline-B**	
Violation rate (S7)	0.0%	0.0%	100.0%	
Availability (S7)	100.0%	100.0%	11.1%	
Never-block (S8)	100.0%	100.0%	0.0%	
S11 (n_s, variant)	**1st min**	**Total**	**Time (s)**	**FP**
3, direct	3	3	0.4	0
5, direct	5	5	0.7	0
8, direct	5	8	80.1	0
12, direct	5	12	150.6	0
12, decoy	5	13	156.4	0
12, L'=12	12	12	0.7	0

RT-only: Runtime-only. 1st min / Total: sources contained (of n_s) within the first minute and in total; Time: median seconds until all sources are contained; FP: addresses blocked outside the campaign. The decoy row counts the twelve decoys and the real source.

In the decoy variant, the twelve decoy addresses consume the per-minute budget for up to three windows, during which a real violation from a further source is contained only after 61 to 151 seconds (median 150 seconds); the per-source cap of three rules in five minutes bounds what any single decoy can consume. Raising the per-minute limit to $L' = 12$ contains all twelve sources within 0.9 seconds, at the cost of admitting up to 2.4 times as many automated rules per minute, and therefore as many wrong rules in a burst, before an operator can intervene. This confirms the intended fail-closed behavior, under which a saturated limit delays containment rather than widening it, and quantifies the containment delay an operator trades for the limit.

6.6. Residual Reasoning and Ablation Analysis (RQ6)

The lower part of Table 10 reports the signature-hint ablation. Scenario S9 contains malicious known-signature cases, malicious unknown-signature cases, and legitimate unknown-signature cases. In BIZM, known signatures are decided by the synthesized program and unknown signatures by the runtime path. On malicious unknown signatures, BIZM is correct in seven of nine decisions and the runtime-only configuration in 28 of 30, whereas SOAR-A is correct in none because it has no matching playbook entry. The runtime path infers maliciousness from context: source and destination zones, policy-matrix entry, connection direction, signature text, and baseline behavior. Both BIZM misses concern one case, held once at confidence 0.66 for review and answered once as log-only, so they fall on the review side rather than on enforcement. On legitimate unknown signatures, both configurations correctly choose no enforcement in all cases.

Scenario S10 injects a three-stage campaign from one source: scanning, application-tier bypass, and database bypass. The synthesized program contains the third stage in all three sequenced and all three cold runs. The runtime-only configuration identifies the multi-stage pattern in all ten runs and cites earlier stages in its reasoning trace, while SOAR-A mitigates only the final signature-matched event. The confidence score does not increase between isolated and correlated presentations because clear violations already saturate near the top of the model's coarse confidence scale.

This ablation should be interpreted narrowly. The synthesized program covers what the knowledge base determines; the runtime path is needed where it does not, such as unknown signatures and alerts that contradict the catalog, and it is less reliable there. The safety invariants remain necessary because the same reasoning component can be wrong or manipulated.

6.7. Algorithm Inputs and Parameter Sensitivity

The inputs of Algorithms 1 and 2 are exercised as follows: the policy P (verdict and conflict check) by S1 to S3, S6, and the sweep; the baseline B (permitted-path judgment) by S4, S5, and S9; the protected set $\mathcal{N}$ by S7, S8, and the sweep; the severity gate ℓ by S7 and RQ1; the history $H(s)$ by S10 and RQ1; the program $\Pi^{\oplus}$ by RQ1, RQ2, and the live scenarios; the synthesis parameters k, $\eta_{\max}$, and T by RQ1 and RQ3; the confidence gates τ_c and τ_Π by S9 and the threshold sweep below; the TTL bounds by the S1 to S4 post-enforcement checks; and the active-rule set $\mathcal{R}$ and the blast-radius limits $\mathcal{L}$ only by S11, because no evaluated decision in S1 to S10 was rejected as a duplicate, by a blast-radius limit, or by the circuit breaker.

The confidence threshold was varied by gating the recorded confidences again: for the runtime path over the 140 runtime-only decisions, and for the compiled path over the 2,510 graded contain points of the deployed program. Raising τ_c from 0.70 to 0.85 changes nothing, because no evaluated runtime-only decision was enforced below 0.88, so the runtime-only TCR stays at 98.9% and the compiled TCR at 100%. Raising it to 0.90 loses the five live threats decided at 0.88, and the runtime-only TCR falls to 93.3%; at 0.95 it falls to 2.2%, and the compiled TCR to 0%, because every DROP rule of the synthesized program carries confidence 0.90, the lowest value among agreeing members, so a threshold above 0.90 requires resynthesizing the program rather than changing a configuration value. The false positive rate stays at zero at every threshold, because the four legitimate cases for which the runtime model proposed a DROP were rejected by the policy-conflict and severity checks, not by the confidence gate.

The TTL bounds do not change any reported outcome: rules became visible within ten seconds of the decision, well inside the 60-second minimum, and all post-enforcement checks passed before expiry. The blast-radius limits bound bursts above five rules per minute or three per source within five minutes, which S1 to S10 do not produce; their effect under a multi-source campaign is reported in Section 6.5.

6.8. Overhead, Cost, and Verifier Scalability (RQ6)

A runtime decision issues one reasoning call and one audit-trace call with a mean prompt of 10.1K tokens, dominated by the retrieved context, or about 21K tokens per

enforced decision. A compiled decision uses no tokens and takes 2 to 17 ms without enforcement; its decision latency on the testbed is the Secure Framework round trip. Synthesis is a one-time cost of 341K tokens and 117 seconds, about the tokens of 16 runtime decisions, repeated only when the knowledge base changes. In the injected runs, the synthesized program answered 45 of the 72 alerts that passed the gate; the runtime path answered the other 27. With the repeats of a case spaced beyond the 60-second lifetime of the decision cache, these runtime-path decisions took a median of 2.9 seconds (95th percentile 18.3 seconds), against a live runtime-only median of 3.1 seconds; the 15-second timeout applies to the enforcement-facing call alone. No reported decision was served from the decision cache: every enforced runtime decision took at least 2.2 seconds, the time of a model call.

Throughput. Under an open-loop offered load of covered-signature alerts, the compiled decision path, which parses the alert, reads the source's alert history from Redis, evaluates the program, builds the intent, and runs the validators, keeps up with 100 alerts per second in a single process with a median of 3.4 ms (99th percentile 47 ms) and no queue growth, and one process sustains about 700 decisions per second. The compiled decision is stateless apart from the alert history, so it parallelizes across processes that share the store: four processes sustain an offered 1,000 alerts per second with a median of 6.0 ms (95th percentile 68 ms, 99th percentile 256 ms), and eight sustain 2,000 per second with a median of 22 ms (99th percentile 699 ms). At that rate the shared history store becomes the bottleneck, with a 99th-percentile lookup of 483 ms, and capacity flattens at 2,300 to 3,100 decisions per second for 4 to 16 processes. Measured through the service, where duplicates within the 30-second suppression window are answered at the gate, the runtime-only configuration keeps up with 0.5 alerts per second (median decision latency 3.8 seconds), answers 0.98 per second at an offered 1 per second with a median rising from 5.1 to 7.1 seconds, and saturates at 1.7 per second at an offered 2 per second, where the median grows from 11.5 to 21.8 seconds within one minute because each non-duplicate alert waits for model calls. The compiled path thus sustains about three orders of magnitude more alerts than the runtime path. These figures are for the decision plane alone; the enforcement path adds the gNMI round trip reported above and is not stressed by alert rate, because duplicate alerts for a covered flow are suppressed while its rule is active.

Verifier scalability. By Proposition 3, the verification cost grows linearly with $|D(K)| \cdot |\Pi|$, and $|D(K)|$ itself grows quadratically in the number of zones and linearly in the number of signatures. For fabrics of $(|Z|, |\Sigma_K|) = (16, 60)$, $(32, 120)$, and $(64, 250)$, the domain has 987,224, 7,379,064, and 59,386,600 points, against 136,080 for the deployed testbed. On synthetic knowledge bases of these sizes, with programs whose length grows with the catalog (43, 85, and 176 rules), single-core verification takes 25.9 and 365 seconds for the first two fabrics, about 0.6 μs per point and rule in both cases, which confirms the bound and projects about 1.7 hours for the largest. With the domain split across signatures, verification takes 6.4 and 54 seconds on 8 cores, and the largest fabric verifies in 537 seconds on 32 cores, with a peak of 90 MB per process; the speed-up of 4.0 and 6.7 on 8 cores flattens beyond that on the shared 32-vCPU host. A synthesis with $k = 5$ and $\eta_{\max} = 3$ issues at most 26 verifications, so fabrics up to 16 zones resynthesize within minutes, whereas the largest listed fabric needs a few hours of offline computation per synthesis unless verification is restricted to the realizable sub-domain $R(K)$, which the IDS catalog fixes through the severity and zone coverage of each signature.

BIZM is intended for containment rather than packet-level inline filtering; high-rate traffic mitigation is better handled by in-network or programmable data-plane mechanisms. Production deployment would also require queueing, overload thresholds, and fail-safe modes for the runtime path, which the throughput experiment characterizes only on the decision plane.

6.9. Discussion, Limitations, and Threats to Validity

The results support three conclusions. First, the LLM contributes most as a synthesizer under verification: a program it derives from the knowledge base agrees with the knowledge base on every graded point, follows its prose where a hand-written rule does not, is resynthesized when the knowledge base changes, and is admitted only after an exhaustive check that rejected three of the six initial candidates. Second, LLM-assisted response can be made operationally useful when the model is not granted direct authority over the network; on both paths, safety comes from deterministic validation and least-privilege enforcement, and on the compiled path it is established on every input in advance. Third, the evaluation provides feasibility evidence on a controlled testbed, not proof of production-scale readiness.

The main limitations are clear. Ground truth is derived from a knowledge base the authors wrote, and the four changes in RQ2 were also authored; a wrong knowledge base yields a program that is consistently wrong, although never unsafe, and the verifier checks safety invariants, not agreement with intent. The program features are coarse, so multi-hop pivot reasoning and payload evidence remain with the runtime path. The testbed is virtualized and small, the scenario set is author-designed, injected alerts do not exercise the IDS sensor, and the cross-model experiment covers two open-weight models. The model confidence is not calibrated; it saturates on clear violations, so the confidence threshold acts as a conservative floor rather than a probabilistic discriminator. The verifier is exhaustive only because the feature abstraction is finite; richer features would require a symbolic verifier, which is a direction for future work rather than a limitation of the guarantee stated here.

There are operational trade-offs. Automatic containment of high-impact assets carries operational risk, which a human-in-the-loop mode for protected assets can address. Blast-radius limits trade containment delay for bounded impact under a coordinated campaign, as S11 quantifies. Target fidelity binds the rule to the header source, so an attacker who can spoof a non-protected address can cause a bounded, time-limited denial of that address's connections to one destination and port; source-address validation at the leaf, which is orthogonal to BIZM, closes this vector, and the per-source and per-minute limits bound it where it remains open. The threat model assumes a correct enforcement implementation and does not cover compromise of the switch operating system, the IDS engine, or encrypted payload inspection beyond what the IDS can observe. A knowledge-base fault-injection experiment, in which seeded errors in the catalog are propagated through synthesis, would further substantiate the claim that a wrong knowledge base yields a consistently wrong but never unsafe program, and is left for future work.

Finally, prior systems are compared by capability rather than by numerical performance (Table 1), because they address different stages of the loop. Baseline-C stands for deterministic context; a learned, anomaly-driven responder was not compared. Schema validation, role-based authorization, TTL-bounded rules, and protected-asset allowlists are established security-engineering mechanisms, and BIZM does not claim them as new. The contribution of BIZM is not a higher detection score but a bounded response architecture that synthesizes verified response logic from the architecture description and connects alert interpretation, safety validation, and least-privilege enforcement in one closed loop.

7. Conclusion

This paper presented BIZM, a safety-constrained framework for bounded isolation in Zero-Trust microsegmentation that treats east-west response as a verification-guided synthesis problem. An LLM synthesizes the architecture knowledge base into a response program of a target-free language; an exhaustive verifier checks the bounded-isolation invariants on every point of the finite alert domain and guides repair with typed counterexamples; and a strict-majority ensemble of verified programs, which we prove remains verified, is deployed and bound to the knowledge base it was synthesized from. At runtime, the LLM reasons only over the alerts the program does not cover. On both paths, enforcement is restricted to source-faithful, time-bounded, provenance-marked DROP rules, and each action must pass fail-closed validation and command-level authorization before reaching the fabric, which bounds the authority of the model even when its output is mistaken, malformed, or manipulated.

The evaluation on a spine-leaf data-center testbed shows that the synthesized program agrees with the knowledge base on all 14,970 graded points, where a hand-written contextual rule reaches 96.1%; that the verifier rejected three of the six initial candidates, which repair corrected within one round; and that resynthesis keeps 99.97% to 100% agreement after four knowledge-base changes. On the live testbed, BIZM contains every threat without a false positive, decides in 0.6 instead of 3.1 seconds, and produces no unsafe enforcement under the tested adversarial alerts, including a coordinated multi-source campaign. These results support the central argument of the paper: LLM-assisted microsegmentation response is feasible when the LLM synthesizes verified response logic from the architecture and its runtime authority is bounded by deterministic network-level safety invariants.

The current prototype has clear limitations. The evaluation uses a controlled virtualized fabric, an author-written knowledge base and scenario set, a coarse program feature set, and two open-weight models. Future work will extend the program features toward multi-hop context with a symbolic verifier, evaluate on larger and more diverse traffic, synthesize from operator-maintained documentation, and develop human-in-the-loop policies for high-impact protected assets and management-plane events.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by the Ministry of Science and Technology of Vietnam under the National Science and Technology Program, project code KC.01.06/21-30.

Data Availability

Data will be made available on request. The testbed configuration, Suricata rules, policy matrix, knowledge base, response-language definition, verifier, synthesis prompts, decision schemas, scenario definitions, evaluation scripts, and raw experimental results can be obtained from the corresponding author.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

The authors used generative AI tools as a coding and writing assistant; they reviewed all content and take full responsibility for it.

CRediT Authorship Contribution Statement

Hanh P. Du: Conceptualization, Formal analysis, Investigation, Methodology, Validation, Writing (original draft). **Hung K. Luu**: Writing (original draft), Writing (review & editing), Visualization, Validation, Software, Investigation, Formal analysis. **Minh T. Nguyen**: Visualization, Validation, Software, Investigation, Formal analysis, Writing (original draft). **Hoa N. Nguyen**: Writing (original draft), Writing (review & editing), Methodology, Conceptualization, Project administration, Supervision.

References

[1] H. R. H. Andersson, A. de Blanche, T. Lundqvist, Flipping the data center network: Increasing east-west capacity using existing hardware, in: Proceedings of the IEEE 42nd Conference on Local Computer Networks (LCN), IEEE, 2017, pp. 211–214. doi:10.1109/LCN.2017.92.

[2] R. Ward, B. Beyer, BeyondCorp: A new approach to enterprise security, USENIX ;login: 39 (6) (2014) 6–11.
URL https://www.usenix.org/system/files/login/articles/login_dec14_02_ward.pdf

[3] S. Rose, O. Borchert, S. Mitchell, S. Connelly, Zero trust architecture, NIST Special Publication 800-207, National Institute of Standards and Technology (2020). doi:10.6028/NIST.SP.800-207.

[4] N. Basta, M. Ikram, M. A. Kaafar, A. Walker, Towards a zero-trust micro-segmentation network security strategy: An evaluation framework, in: Proceedings of the 2022 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2022, pp. 1–7. doi:10.1109/NOMS54207.2022.9789888.

[5] C. Buck, C. Olenberger, A. Schweizer, F. Völter, T. Eymann, Never trust, always verify: A multivocal literature review on current knowledge and research gaps of zero-trust, Computers & Security 110 (2021) 102436. doi:10.1016/j.cose.2021.102436.

[6] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, R. Doss, Zero trust architecture (ZTA): A comprehensive survey, IEEE Access 10 (2022) 57143–57179. doi:10.1109/ACCESS.2022.3174679.

[7] R. Chandramouli, A zero trust architecture model for access control in cloud-native applications in multi-location environments, NIST Special Publication 800-207A, National Institute of Standards and Technology (2023). doi:10.6028/NIST.SP.800-207A.

[8] R. Chandramouli, Building secure microservices-based applications using service-mesh architecture, NIST Special Publication 800-204A, National Institute of Standards and Technology (2020). doi:10.6028/NIST.SP.800-204A.

[9] K. Hsieh, M. Wong, S. Segarra, S. K. Mani, T. Eberl, A. Panasyuk, R. Netravali, R. Chandra, S. Kandula, NetVigil: Robust and low-cost anomaly detection for east-west data center security, in: Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2024, pp. 1771–1780.
URL https://www.usenix.org/conference/nsdi24/presentation/hsieh

[10] I. J. King, H. H. Huang, Euler: Detecting network lateral movement via scalable temporal graph link prediction, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2022.
URL https://www.ndss-symposium.org/ndss-paper/euler-detecting-network-lateral-movement-via-scalable-temporal-graph-link-prediction/

[11] Open Information Security Foundation, Suricata: Open source threat detection engine, https://suricata.io/, accessed: 2026-06-04 (2026).

[12] H. V. Vo, H. P. Du, H. N. Nguyen, APELID: Enhancing real-time intrusion detection with augmented WGAN and parallel ensemble learning, Computers & Security 136 (2024) 103567. doi:10.1016/j.cose.2023.103567.

[13] T. T. Nguyen, U. H. Tran, H. N. Nguyen, Adversarial attack and defense in AI-powered intrusion detection, Journal of Computer Science and Cybernetics 41 (4) (2025) 387–402. doi:10.15625/1813-9663/22884.
URL https://vjst.vast.vn/jcc/article/view/22884

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837.
URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: Proceedings of the 11th International Conference on Learning Representations (ICLR), 2023.
URL https://openreview.net/forum?id=WE_vluYUL-X

[16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.
URL https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[17] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[18] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: Proceedings of the 11th International Conference on Learning Representations (ICLR), 2023.
URL https://openreview.net/forum?id=1PL1NIMMrw

[19] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, H. Hajishirzi, P. Clark, Self-refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[20] Y. Liu, T. Han, S. Ma, J. Zhang, Y. Yang, J. Tian, H. He, A. Li, M. He, Z. Liu, Z. Wu, L. Zhu, D. Li, N. Qiang, D. Shen, T. Liu, B. Ge, Summary of ChatGPT-related research and perspective towards the future of large language models, Meta-Radiology 1 (2) (2023) 100017. doi:10.1016/j.metrad.2023.100017.

[21] X. Yue, B. Wang, K. Li, Z. Su, H. Sun, Automatic evaluation of attribution by large language models, in: Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 4615–4635. doi:10.18653/v1/2023.findings-emnlp.307.

[22] Y. Liu, Y. Jia, R. Geng, J. Jia, N. Z. Gong, Formalizing and benchmarking prompt injection attacks and defenses, in: Proceedings of the 33rd USENIX Security Symposium, USENIX Association, 2024, pp. 1831–1847.
URL https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[23] S. K. Mani, K. Hsieh, S. Segarra, R. Chandra, Y. Zhou, S. Kandula, Securing public cloud networks with efficient role-based micro-segmentation, in: Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2025, pp. 1033–1048.
URL https://www.usenix.org/conference/nsdi25/presentation/mani

[24] D. Canavese, R. Laborde, A. Laraba, A. Ferreira, A. Benzekri, Game of zones: An automated intent-based network micro-segmentation methodology, in: Proceedings of the 2025 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2025, pp. 1–9. doi:10.1109/NOMS57970.2025.11073630.

[25] C. Katsis, E. Bertino, ZT-SDN: An ML-powered zero-trust architecture for software-defined networks, ACM Transactions on Privacy and Security 28 (2) (2025). doi:10.1145/3712262.

[26] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, M. Chiesa, NetConfEval: Can LLMs facilitate network configuration?, Proceedings of the ACM on Networking 2 (CoNEXT2) (2024) 1–25. doi:10.1145/3656296.

[27] A. Angi, A. Sacco, G. Marchetto, LLNet: An intent-driven approach to instructing softwarized network devices using a small language model, IEEE Transactions on Network and Service Management 22 (4) (2025) 3403–3418. doi:10.1109/TNSM.2025.3570017.

[28] R. Mondal, A. Tang, R. Beckett, T. Millstein, G. Varghese, What do LLMs need to synthesize correct router configurations?, in: Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets), ACM, 2023, pp. 189–195. doi:10.1145/3626111.3628194.

[29] L. Mhamdi, M. M. Isa, Securing SDN: Hybrid autoencoder-random forest for intrusion detection and attack mitigation, Journal of Network and Computer Applications 225 (2024) 103868. doi:10.1016/j.jnca.2024.103868.

[30] L. D. Manocchio, Y. Chen, S. Layeghy, D. Gwynne, M. Portmann, P4-Secure: In-band DDoS detection in software defined networks, IEEE Transactions on Network and Service Management 22 (2) (2025) 2120–2137. doi:10.1109/TNSM.2025.3552844.

[31] Y. Bello, A. R. Hussein, Dynamic policy decision/enforcement security zoning through stochastic games and meta learning, IEEE Transactions on Network and Service Management 22 (1) (2025) 807–821. doi:10.1109/TNSM.2024.3481662.

[32] N. N. Hari, P. Krishnan, K. Jain, A. K. J. Saudagar, P. Prabu, R. C. Poonia, A three-tier microsegmentation framework for enterprise networks under zero trust architecture, Alexandria Engineering Journal 141 (2026) 150–164. doi:10.1016/j.aej.2026.03.014.

[33] S. G. E. Gökstorp, J. Nyberg, Y. Kim, P. Johnson, G. Dán, Anomaly detection in security logs using sequence modeling, in: Proceedings of the 2024 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2024, pp. 1–9. doi:10.1109/NOMS59830.2024.10575561.

[34] H. V. Vo, H. N. Nguyen, T. N. Nguyen, H. P. Du, SDAID: Towards a hybrid signature and deep analysis-based intrusion detection method, in: Proceedings of the 2022 IEEE Global Communications Conference (GLOBECOM), IEEE, 2022, pp. 2615–2620. doi:10.1109/GLOBECOM48099.2022.10001582.

[35] H. V. Vo, H. P. Du, H. N. Nguyen, AI-powered intrusion detection in large-scale traffic networks based on flow sensing strategy and parallel deep analysis, Journal of Network and Computer Applications 220 (2023) 103735. doi:10.1016/j.jnca.2023.103735.

[36] K. Dzeparoska, J. Lin, A. Tizghadam, A. Leon-Garcia, LLM-based policy generation for intent-based management of applications, in: Proceedings of the 2023 19th International Conference on Network and Service Management (CNSM), IEEE, 2023, pp. 1–7. doi:10.23919/CNSM59352.2023.10327837.

[37] M. Siino, F. Giuliano, I. Tinnirello, Integrating large language models into network testbeds: A novel approach for automated experimentation and optimization, Journal of Network and Computer Applications 243 (2025) 104281. doi:10.1016/j.jnca.2025.104281.

[38] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia, V. Saraswat, Combinatorial sketching for finite programs, in: Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2006, pp. 404–415. doi:10.1145/1168857.1168907.

[39] S. Gulwani, O. Polozov, R. Singh, Program synthesis, Foundations and Trends in Programming Languages 4 (1–2) (2017) 1–119. doi:10.1561/2500000010.

[40] P. Kazemian, G. Varghese, N. McKeown, Header space analysis: Static checking for networks, in: Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2012, pp. 113–126.
URL https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/kazemian

[41] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, T. Millstein, A general approach to network configuration analysis, in: Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2015, pp. 469–483.
URL https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/fogel

[42] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, in: Advances in Neural Information Processing Systems, Vol. 30, 2017.
URL https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

[43] P. Cichonski, T. Millar, T. Grance, K. Scarfone, Computer security incident handling guide, NIST Special Publication 800-61 Rev. 2, National Institute of Standards and Technology (2012). doi:10.6028/NIST.SP.800-61r2.

[44] A. Nelson, M. Souppaya, K. Scarfone, Incident response recommendations and considerations for cybersecurity risk management, NIST Special Publication 800-61 Rev. 3, National Institute of Standards and Technology (2025). doi:10.6028/NIST.SP.800-61r3.

[45] I. Belhaj, Q. Zhang, C. Alcaraz, J. Lopez, Cybersecurity automation and orchestration: A survey, ACM Computing Surveys 56 (2) (2023) 1–36. doi:10.1145/3605776.

[46] H. P. Du, M. Q. Tran, T. T. Nguyen, H. N. Nguyen, Multi-layer defense for AI-powered IDS: Ensemble adversarial training and explainable resilience to evasion attacks, in: Proceedings of the 2025 2nd International Conference on Cryptography and Information Security (VCRIS), IEEE, 2025, pp. 1–6. doi:10.1109/VCRIS68011.2025.11250565.

[47] H. V. Vo, H. P. Du, H. N. Nguyen, MDOB: Enhancing resilient and explainable AI-powered malware detection using feature set optimization and mutual deep+boosting ensemble inference, Journal of Information Security and Applications 93 (2025) 104175. doi:10.1016/j.jisa.2025.104175.
URL https://www.sciencedirect.com/science/article/pii/S2214212625002121

[48] H. V. Le, T. N. Nguyen, H. N. Nguyen, L. Le, An efficient hybrid webshell detection method for webserver of marine transportation systems, IEEE Transactions on Intelligent Transportation Systems 24 (2) (2023) 2630–2642. doi:10.1109/TITS.2021.3122979.

[49] R. Doriguzzi-Corin, L. A. D. Knob, L. Mendozzi, D. Siracusa, M. Savi, Introducing packet-level analysis in programmable data planes to advance network intrusion detection, Computer Networks 239 (2024) 110162. doi:10.1016/j.comnet.2023.110162.

[50] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, OpenFlow: Enabling innovation in campus networks, ACM SIGCOMM Computer Communication Review 38 (2) (2008) 69–74. doi:10.1145/1355734.1355746.

[51] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, D. Walker, P4: Programming protocol-independent packet processors, ACM SIGCOMM Computer Communication Review 44 (3) (2014) 87–95. doi:10.1145/2656877.2656890.

[52] H. Farhady, H. Lee, A. Nakao, Software-defined networking: A survey, Computer Networks 81 (2015) 79–95. doi:10.1016/j.comnet.2015.02.014.

[53] C. Jung, M. Ghasemi, T. Benson, J. Rexford, A scalable and dynamic ACL system for in-network defense, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2022, pp. 231–245. doi:10.1145/3548606.3560606.

[54] Y.-D. Lin, H.-X. Huang, D. Sudyana, Y.-C. Lai, AI for AI-based intrusion detection as a service: Reinforcement learning to configure models, tasks, and capacities, Journal of Network and Computer Applications 229 (2024) 103936. doi:10.1016/j.jnca.2024.103936.

[55] F. Metzger, R. Irmer, I. Grønbæk, K. Tutschku, Intent-based networking: Concepts and definitions, in: Proceedings of the 2023 IEEE Conference on Network Softwarization (NetSoft), IEEE, 2023, pp. 1–7. doi:10.1109/NetSoft57336.2023.10175424.

[56] D. D. A. Nguyen, F. Autrel, A. Bouabdallah, G. Doyen, A robust approach for the detection and prevention of conflicts in I2NSF security policies, in: Proceedings of the 2023 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2023, pp. 1–7. doi:10.1109/NOMS56928.2023.10154304.

[57] D. Bringhenti, G. Marchetto, R. Sisto, F. Valenza, J. Yusupov, Automated firewall configuration in virtual networks, IEEE Transactions on Dependable and Secure Computing 20 (2) (2023) 1559–1576. doi:10.1109/TDSC.2022.3160293.

[58] W. Zahwa, A. Lahmadi, M. Rusinowitch, M. Ayadi, Deep reinforcement learning for in-network placement of ACL rules under constraints, in: Proceedings of the 2025 21st International Conference on Network and Service Management (CNSM), IEEE, 2025, pp. 1–7. doi:10.23919/CNSM67658.2025.11297518.

[59] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, J. Cohen, NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, 2023, pp. 431–445. doi:10.18653/v1/2023.emnlp-demo.40.

[60] S. Chen, J. Piet, C. Sitawarin, D. Wagner, StruQ: Defending against prompt injection with structured queries, in: Proceedings of the 34th USENIX Security Symposium, USENIX Association, 2025, pp. 2383–2400.
URL https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe

[61] Y. Wu, F. Roesner, T. Kohno, N. Zhang, U. Iqbal, IsolateGPT: An execution isolation architecture for LLM-based agentic systems, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2025.
URL https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/

[62] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, C. B. Thomas, MITRE ATT&CK: Design and philosophy, Tech. Rep. MP180360, The MITRE Corporation (2018).
URL https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

[63] MITRE Corporation, MITRE ATT&CK: Enterprise matrix, https://attack.mitre.org/, accessed: 2026-06-04 (2026).

[64] Linux Foundation, SONiC: Software for open networking in the cloud, https://sonicfoundation.dev/, accessed: 2026-06-04 (2026).